

A-

COMP 361D2: Specification Document

Progress Report 2

“OpenRobotics”: Humanoid Robot Warehouse Simulator

January 21, 2026

Murad Novruzov, Dan Moraru, Daniel Blackburn,
Lounes Azzoun, Muhammed Sohail,
Badr Zeji, Filip Snítíl

Table of Contents

1. Background	6
1.1 Motivation	6
1.2 Stakeholders	6
Table 1.2.1 Stakeholders Table	6
2. Overview	8
2.1 Usage	8
2.1.1 Usage Table	8
2.1.2 Access Descriptions	8
2.2 Project	10
2.2.1 Scheduling	10
2.2.2 Project Milestones and Deliverables	10
Table 2.2.3 Sprint Schedule with Course Milestones	11
2.2.4 Open Robotics Warehouse Simulator Storyboard	13
2.2.5 Diagram Key	13
2.2.6 Storyboard Component Descriptions	14
2.2.7 Deployment Diagram	16
3. Definitions	19
3.1 Branding	19
3.1.1 Visual Identity & Feel	19
3.1.2 Logo	19
Figure 3.1.2 A: Logo in a light background	20
Figure 3.1.2 B: Logo in a dark background	20
3.1.3 Color Palette	21
3.2 Required Technologies	22
3.2.1 Devices and Hardware Requirements	22
3.2.2 Programming Languages, Libraries, and Frameworks	22
3.2.3 Runtime Environments	23
3.2.4 Compute, Network, and Server Requirements	23
3.3 Project and System Constraints	24
3.3.1 Budget Constraints	24
3.3.2 Equipment Constraints	24
3.3.3 Technical Constraints	24
3.3.4 Scope Constraints	24
3.4 Client Expectations	25
3.4.1 Must-Have Features	25
3.4.2 Wants / Nice-to-Haves	25
3.4.3 Assumptions	25
3.4.4 Success and Failure Criteria	26
3.4.5 Explicit Scope Statements	26

3.5 Impact, Ethics, and Performance Statements	27
3.5.1 Impact Statement	27
3.5.2 Ethical Considerations	27
3.5.3 Performance Statements	27
4. Screen Layouts	28
4.1 Screen Mockups	28
4.1.1 Welcome Screen	28
Figure 4.1.1 : Welcome Screen Mockup	28
4.1.2 Setup Screen	29
Figure 4.1.2 : Setup Screen Mockup	29
4.1.3 Simulation Screen	30
Figure 4.1.3 A: Simulation Screen Mockup	30
Figure 4.1.3 B: Outliner Window Mockup	30
4.1.3 Results Screen	32
Figure 4.1.3: Results Screen Mockup	32
4.2 Specialty Screens	33
Figure 4.2 : Specialty Screens	33
5. Sections & Queries	34
5.1 Defining File Structures	34
5.1.1 Database Diagram	34
5.1.2 Database table structures	35
5.1.2.1 Maps Table	35
5.1.2.2 Simulation_Runs Table	35
5.1.2.3 Run_Results Table	36
5.1.2.4 Sim_Logs Table	37
5.1.2.5 Robot_Run_Stats Table	37
5.1.2.6 Run_Workload_Tasks Table	38
5.1.3 Formatted text files	39
5.1.3.1 CSV Export: results_runs.csv	39
5.1.3.2 CSV Export: results_robots.csv	39
5.1.3.3 JSON File: map.json	39
5.1.3.4 JSON File : run_config.json	40
5.1.4 Defined Operations	40
5.1.4.1 Maps Table Queries	40
5.1.4.2 Simulation_Runs Table Queries	41
5.1.4.3 Run_Results Table Queries	41
5.1.4.4 Sim_Logs Table Queries	42
5.1.4.5 Robot_Run_Stats Table Queries	42
5.1.4.6 Run_Workload_Tasks Table Queries	42
5.2 Remote API definitions	43
5.2.1 Global API Rules (Applies to All Endpoints)	43

5.2.2 Maps API	44
5.2.2.1 API Call Signatures	44
5.2.2.2 GET /maps/list	44
5.2.2.3 GET /maps/get/{mapId}	45
5.2.2.4 GET /maps/search?name={substring}	46
5.2.2.5 POST /maps/create	47
5.2.3 Runs API	47
5.2.3.1 API Call Signatures	47
5.2.3.2 POST /runs/create	48
5.2.3.3 POST /runs/start/{runId}	48
5.2.3.4 POST /runs/stop/{runId}	49
5.2.3.5 GET /runs/status/{runId}	49
5.2.3.6 GET /runs/list	50
5.2.3.7 GET /runs/search?mapId={uuid}&robotCount={int}&policy={...}	51
5.2.4 Results API	51
5.2.4.1 API Call Signatures	51
5.2.4.2 GET /results/run/{runId}	51
5.2.4.3 GET /results/robots/{runId}	52
5.2.4.4 GET /results/compare?mapId={uuid}&policy={...}	53
5.2.5 Logs API	54
5.2.5.1 API Call Signatures	54
5.2.5.2 GET /logs/run/{runId}	54
5.2.5.3 GET /logs/search?runId={uuid}&eventType={list}&robotId={int}&fromTick={int}&toTick={int}	55
5.2.6 Exports API (CSV)	56
5.2.6.1 API Call Signatures	56
5.2.6.2 GET /export/results/{runId}.csv	56
5.2.6.3 GET /export/robots/{runId}.csv	56
5.2.6.4 GET /export/heatmap/{runId}.csv	57
6. Critical Sections	58
6.1 Critical sections table of contents	58
6.2 Critical Sections Breakdown	59
6.2.1 Critical Section 1 – Tick Order (Per-Tick Simulation Update Pipeline)	59
6.2.2 Critical Section 2 – Move Intentions & Collision/Conflict Resolution	61
6.2.3 Critical Section 3 – Task Assignment (Dispatcher)	63
6.2.4 Critical Section 4 – Traffic Rules Coordination Policy	65
6.2.5 Critical Section 5 – Tile Reservation Coordination Policy	67
6.2.6 Critical Section 6 – Deadlock Detection & Recovery	69
6.2.7 Critical Section 7 – Greedy Navigation Strategy	71
7. Effort estimation	73

Try to not
have these
hanging titles.

7.1 Estimation Approach	73
7.2 Effort Estimation Graph	73
7.3 Gantt Chart	74
7.4 Estimated Project Cost	75
8. Resources	75
Appendix A: Teams Contribution Table	75
Appendix B: Links & References	75

1. Background

1.1 Motivation

The Humanoid Robot Warehouse Simulator is a software-based simulation platform designed to model, analyze, and evaluate the behavior of autonomous humanoid robots operating within a warehouse environment. The simulator represents a warehouse as a structured grid of aisles, storage locations, and intersections, in which multiple robots navigate concurrently to complete assigned tasks such as moving between locations to transport items. By simulating robot movement, navigation strategies, and interactions, the system provides a high-level yet realistic environment for studying autonomous warehouse operations without reliance on physical hardware.

The primary goal of this project is to support research into how warehouse workloads can be completed efficiently and reliably using teams of robots. This tool will provide a controlled simulation for multi-robot warehouse systems, enabling the study of trade-offs among fleet size, throughput, task distribution, and workload completion time. By isolating algorithmic behaviour from hardware differences, the simulator enables repeatable, data-driven experiments that support evaluation of coordination and navigation design decisions.

This tool is intended for organizations involved in warehouse logistics that are considering or planning a transition to robotic systems. It addresses their need to evaluate design choices such as fleet size and control strategies without the cost and risk of physical deployment. By enabling experimentation in simulation, the tool can help reduce operational costs while supporting more informed decision-making.

1.2 Stakeholders

Table 1.2.1 Stakeholders Table

Stakeholder Type	Name/Role	Coordinates	Success Measure
Client/Owner	Prof. Vybihal	joseph.vybihal@mcgill.ca	The application works without any major/critical bugs and is intuitive to use. The final application is completed on time.
End User (Primary)	Researcher/Student (Team Members)	N/A	The application works without any major/critical bugs,

			functions as intended, and is a useful tool for research. The final application is completed on time.
End User (Secondary)	Evaluator/TA	N/A	The application works without any major/critical bugs, is intuitive to use, was created following the guidelines of the project rubric, and is original (i.e. not plagiarized)
Indirect User	Researchers, Warehouse Managers, Educators, Warehouse customers	N/A	The application can be used to find optimal ways to manage the workload of a multi-robot warehouse system to save time and money.

Good

2. Overview

2.1 Usage

Always provide at least a sentence or a short paragraph for every point number, for example, 2.1 and then 2.1.1 before or after the table.

2.1.1 Usage Table

Stakeholder Type	Name/Role	When They Use It	Frequency	Features Accessed
Owner	Prof. Vybihal	Progress demos (Mar 13, Mar 31, Apr 14), Acceptance test (Apr 16-30), Work checkups	5-6 times total	View simulation runs, review results dashboard, verify all features work, test edge cases
End User (Primary)	Researcher/Student (Team Members)	Development & testing phase (Jan-Apr), Running experiments to compare algorithms	Daily during development	All features: configure simulations, select maps/algorithms/coordination policies, run simulations, analyze metrics, save/load configurations
End User (Secondary)	Evaluator/TA	Grading demos, Acceptance test	3-4 times total	Run pre-configured simulations, observe simulation visualization, review results and metrics, verify functionality without assistance
Indirect User	Researchers, Warehouse Managers, Educators Warehouse customers	After experiments are completed and findings are compiled	N/A - does not interact with application	None - consumes exported results and reports generated from simulation data

Good table.

2.1.2 Access Descriptions

Professor Vybihal (Owner)

The professor accesses the simulator during scheduled progress demonstrations and the final acceptance test. At demos, a team member launches the application and presents the current state of development. The professor observes the simulation running, asks questions about specific features, and may request to see particular scenarios such as collision handling or

deadlock recovery. During the acceptance test, the professor or a designated grader launches the application independently on the team's laptop and operates it without assistance from team members. The professor navigates from the welcome screen, configures a simulation by selecting a map and robot parameters, runs the simulation, and reviews the results dashboard to verify correctness and completeness.

Researcher/Student (Primary End User)

Team members are the primary users throughout development. A researcher launches the application, which opens the welcome screen. From there, the user proceeds to the simulation setup screen where they select a map (preset or randomized), specify the number of robots and their assigned navigation algorithms, choose a coordination policy, and configure workload parameters such as task count and spawn rate. The user can save this configuration for later reuse or load a previously saved setup. Once configured, the user starts the simulation and observes the robots executing tasks in real-time through the visualization. The user can pause, resume, or stop the simulation at any time. After completion, the user accesses the results dashboard to review throughput metrics, collision counts, energy consumption, and path heatmaps. Past experiment results are stored automatically, allowing the user to return later and compare performance across multiple runs.

Evaluator/TA (Secondary End User)

The TA accesses the simulator during grading sessions for progress demos and the acceptance test. The TA launches the application and navigates through the interface to verify that required features are functional. A typical session involves loading a preset map, running a simulation with default parameters, and confirming that the visualization displays robot movement correctly. The TA reviews the results dashboard to check that metrics are calculated and displayed properly. During the acceptance test specifically, the TA operates the simulator without guidance, following an intuitive interface to test various configurations and verify that the program does not crash under normal usage.

Researchers, Warehouse Managers, and Educators (Indirect Users)

Indirect users do not interact with the simulator directly. Instead, they benefit from the research findings and data produced by primary users who run experiments. A warehouse manager might review exported results to inform purchasing decisions about fleet size or to evaluate whether certain navigation strategies would work in their facility layout. Researchers in robotics or logistics could use the findings as a baseline for further study or to validate their own models. Educators might reference the experimental data when teaching concepts related to multi-robot coordination, pathfinding algorithms, or warehouse automation. These users receive information through exported reports, presentations, or documentation produced by the team rather than through the application interface itself.

Okay.

2.2 Project

2.2.1 Scheduling

Expected Development Timeline

Development for the Open Robotics Warehouse Simulator is scheduled to begin on January 5, 2026, following the submission of the Specification Document. The project is expected to be completed by April 14, 2026, with the final demonstration. The total development period spans approximately 14 weeks, organized into seven two-week sprints following an Agile workflow.

Major Milestones

Good to explain how to understand the sprint to milestones alignment in some words.

2.2.2 Project Milestones and Deliverables

Date	Milestone	Deliverable	Weight
Dec. 1, 2025	Team Registration	Registration form submitted	P/F
Jan. 21, 2026	Specification Document	Complete specification document (this document)	5%
Jan. 28, 2026	Work Checkup	Meeting with TA/professor to review specification plan	P/F
Feb. 11, 2026	Design Document	Complete design document with UML diagrams, class structures, and technical specifications	10%
Mar. 13, 2026	Interim Demo 1	Program ~1/3 complete; skeleton MVP with basic robot movement and map rendering functional	2.5%

Technically, this is the course's milestones.

Mar. 16, 2026	Failsafe Checkup	Meeting with the professor to address any team or development issues	P/F
Mar. 31, 2026	Interim Demo 2	Program ~1/2 complete; navigation and coordination systems functional	2.5%
Apr. 14, 2026	Final Demonstration	Near-complete program with all core features implemented and tested	10%
Apr. 16–30, 2026	Acceptance Test	Complete, intuitive, crash-free application ready for independent evaluation	15%
Apr. 30, 2026	Scrum Journal	Individual reflection journals documenting team experience	5%

Sprint-to-Milestone Alignment

Table 2.2.3 Sprint Schedule with Course Milestones

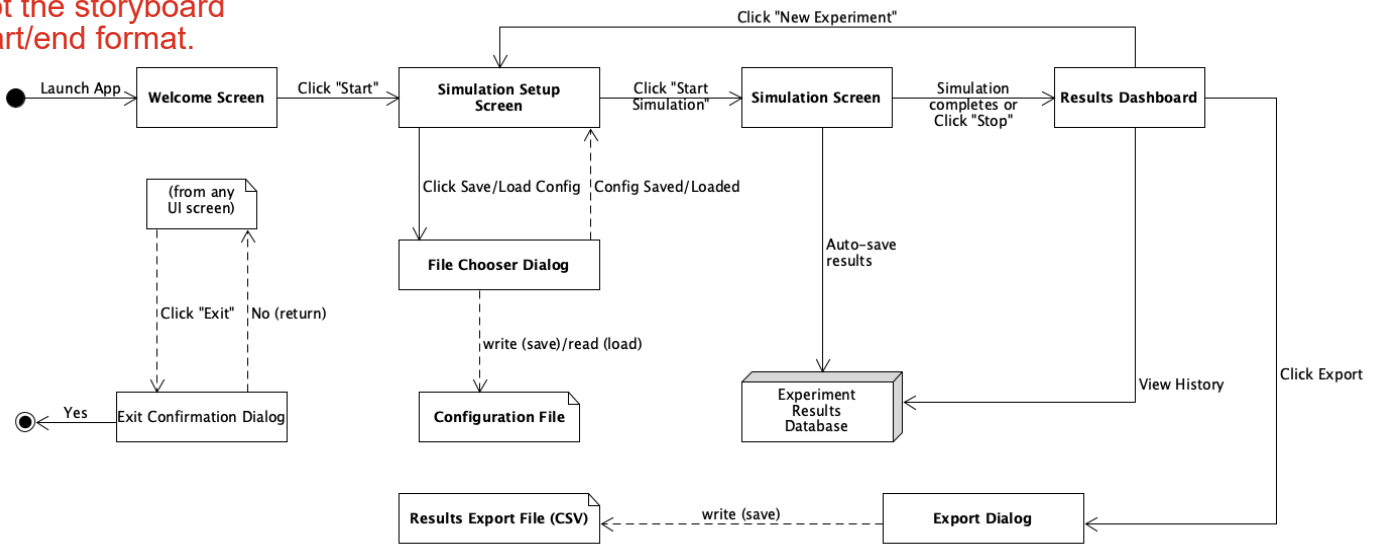
Sprint	Dates	Focus	Milestone
Sprint 1	Jan. 5 – Jan. 18	Specification finalization, environment setup	Specification Doc (Jan. 21)
Sprint 2	Jan. 19 – Feb. 1	Design document, skeleton code structure	Work Checkup (Jan. 28)

Sprint 3	Feb. 2 – Feb. 15	Skeleton MVP: single robot, basic map, UI shell	Design Doc (Feb. 11)
Sprint 4	Feb. 16 – Mar. 1	Navigation MVP: sensors, algorithms, robot movement	—
Sprint 5	Mar. 2 – Mar. 15	Coordination MVP: dispatcher, task system, policies	Interim Demo 1 (Mar. 13)
Sprint 6	Mar. 16 – Mar. 29	Analytical MVP: metrics, results dashboard, database	Failsafe Checkup (Mar. 16)
Sprint 7	Mar. 30 – Apr. 14	Final push: optimization, polish, bug fixes, stretch goals	Interim Demo 2 (Mar. 31), Final Demo (Apr. 14)

Okay, this is a good mapping.

2.2.4 Open Robotics Warehouse Simulator Storyboard

Not the storyboard start/end format.



2.2.5 Diagram Key

You don't need 2.2.5 for the diagram key because the key is still about 2.2.4.

Symbol	Element Name	Description
Solid Circle	Entry Point	The starting point of the application flow where the user launches the application.
Bullseye Circle	Termination Point	The final end state where the application closes completely.
Rectangle	UI Screen / Dialog	Represents a specific window or dialog box in the application where the user interacts with the system (e.g., <i>Simulation Setup Screen</i>).
3D Box (Node)	Database	Represents the persistent storage system where experiment results and history are stored and retrieved.
Document Icon	Configuration File	Represents external files used to save or load custom simulation settings.
Folded Corner Box	Global Context / Note Box	Represents a rule or action that applies globally across the application, rather than starting from a single specific screen (e.g., the ability to exit the app from anywhere).

Solid Arrow	Link	Indicates a direct transition from one screen to another, triggered by a user action or system event.
Dashed Arrow	Redirect	Represents a transition that can occur from multiple contexts rather than a direct link from one specific screen.
Labeled Data Arrow	Data Operation	Represents an instantaneous read or write operation between a dialog and an external file or database.

2.2.6 Storyboard Component Descriptions

Welcome Screen

The welcome screen is the first screen users see when launching the application. It displays the application logo and branding, and provides a single entry point into the simulator. The user clicks the "Start" button to proceed to the simulation setup screen. This screen serves as a simple introduction and ensures the application has fully loaded before the user begins configuring their experiment.

Simulation Setup Screen

The simulation setup screen is the central hub where users configure all parameters for their experiment. Users can select or generate a map, specify the number of robots, assign navigation algorithms to each robot, choose a coordination policy, and set workload parameters such as task count and spawn rate. From this screen, users can optionally save their current configuration to a file for later reuse or load a previously saved configuration. Once satisfied with the setup, users click "Start Simulation" to begin the experiment.

File Chooser Dialog and Configuration File

The file chooser dialog appears when users click "Save" or "Load" from the simulation setup screen. When saving, the dialog allows users to specify a filename and location, and the current setup parameters are written to a configuration file instantaneously. When loading, users select an existing configuration file, which is read immediately and populates the setup screen with the saved values, which they can review before starting. After either operation completes, users are returned to the simulation setup screen. The configuration file is entirely optional and exists for the convenience and reproducibility of experiments.

Simulation Screen

The simulation screen displays the warehouse environment and robots in real time as the experiment runs. Users can observe robots navigating the map, picking up items, delivering to packing stations, and charging when needed. Playback controls allow users to pause, resume, or stop the simulation, as well as adjust the simulation speed (x1, x2, x3, etc.) so users don't have to watch lengthy experiments in real time. The screen also displays live statistics such as tasks completed, active collisions, and elapsed time. The simulation ends either when all tasks are completed, when the maximum tick count is reached, or when the user manually clicks "Stop."

Results Dashboard and Experiment Results Database

The results dashboard presents a summary of the completed experiment including throughput metrics, collision counts, energy consumption, and per-robot statistics. Results are automatically saved to the database when the simulation ends. Users can also access historical experiment data by clicking "View History," which retrieves past runs from the database for comparison. From this screen, users can click "New Experiment" to return to the simulation setup screen and configure another run.

Export Dialog and Results Export File

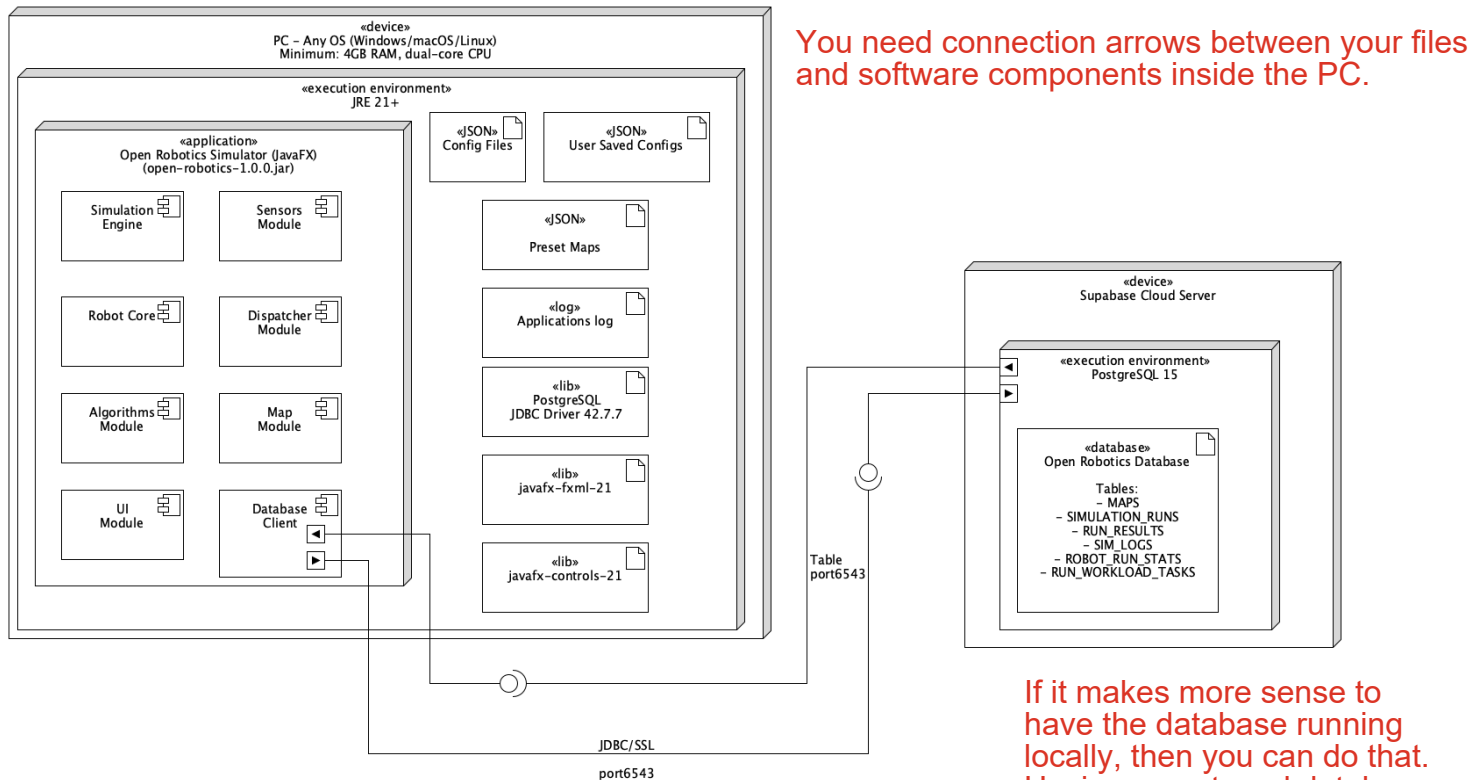
The export dialog is triggered when users click "Export" from the results dashboard. This opens the system file explorer, allowing users to select a directory and specify a filename for the exported data. Upon confirmation, the application writes the experiment results to a CSV file at the chosen location. This export includes all metrics from the current experiment such as throughput, collision counts, energy consumption, and per-robot statistics, formatted for easy analysis in spreadsheet software like Excel. The export feature is separate from the automatic database save and provides users with a portable file they can share or analyze outside the application.

Exit Confirmation Dialog

The exit confirmation dialog can be triggered from any screen in the application by clicking the "Exit" button, represented by the dashed redirect arrow from the global context. This dialog asks users to confirm they want to close the application. Clicking "No" returns users to whatever screen they were previously on, while clicking "Yes" terminates the application and reaches the final end state.

2.2.7 Deployment Diagram

Open Robotics Warehouse Simulator Deployment Diagram



Client Device (PC)

The client device represents the end user's personal computer where the simulator application runs. The system supports Windows, macOS, and Linux operating systems, requiring a minimum of 4GB RAM and a dual-core CPU. These specifications ensure smooth simulation rendering and real-time visualization of robot movements. Users launch the application on their local machine, and all simulation processing occurs client-side.

Java Runtime Environment (JRE 21+)

The Java Runtime Environment version 21 or higher serves as the execution environment for the application. JRE provides the virtual machine that interprets the compiled Java bytecode and manages memory allocation during simulation runs. The choice of Java 21+ ensures access to modern language features and long-term support, while maintaining cross-platform compatibility across all supported operating systems.

Open Robotics Simulator Application

The main application is packaged as a JAR file (open-robotics-1.0.0.jar) built with JavaFX for the graphical user interface. The application contains eight internal modules: Simulation Engine manages the tick-based simulation loop; Sensors Module handles robot perception; Robot Core implements robot state machines and energy models; Dispatcher Module assigns tasks to robots; Algorithms Module contains navigation strategies; Map Module manages warehouse layouts; Database Client handles communication with the remote database; and UI Module renders all screens and visualizations.

Configuration Files

Two types of JSON configuration files support the application. Config Files stored in the ./config/ directory contain default simulation parameters that ship with the application. User Saved Configs stored in ~/.open-robotics/ preserve custom simulation setups that users create and save for later reuse. This separation allows the application to maintain factory defaults while enabling users to build a personal library of experiment configurations.

Preset Maps

Preset maps are JSON files bundled within the application JAR in the resources folder. These include three stress-test scenarios: a small baseline map for demonstrations, a narrow aisles map for testing tight navigation, and a many intersections map for evaluating coordination policies. Since they are packaged inside the JAR, preset maps cannot be modified by users and remain consistent across all installations.

Applications Log

The applications log file records runtime events, errors, and diagnostic information during simulation execution. This log supports debugging during development and provides a record of any issues that occur during experiments. The log file is stored locally on the client machine.

Runtime Libraries

Three external libraries support the application. The PostgreSQL JDBC Driver version 42.7.7 enables database connectivity using the JDBC protocol. JavaFX-fxml-21 provides the framework for loading screen layouts from FXML files. JavaFX-controls-21 supplies the UI components including buttons, tables, and input fields used throughout the interface.

Supabase Cloud Server

The Supabase Cloud Server provides managed database hosting for the project. Supabase was selected because it offers a PostgreSQL database with a free tier sufficient for course project needs, allows all team members to access shared experiment data, and eliminates the need to host and maintain a dedicated database server. The managed service handles backups, scaling, and availability automatically.

Ya, but does it make sense? It is not yet clear from your feature description. It may be easier to build but more expensive for the user to run, so you need to have a justification that gives value to the client.

PostgreSQL 15 Execution Environment

PostgreSQL version 15 runs as the database management system on the Supabase server. This execution environment processes SQL queries, manages transactions, and enforces data integrity constraints. PostgreSQL was chosen for its reliability, SQL compliance, and native compatibility with the JDBC driver already included in the project.

Open Robotics Database

The database stores all persistent data for the simulator across six tables. MAPS stores warehouse layouts including tile data and dimensions. SIMULATION_RUNS records the configuration of each experiment including robot count, algorithms, and coordination policy. RUN_RESULTS captures aggregate metrics like throughput, collisions, and energy consumption. SIM_LOGS stores detailed tick-by-tick event data for post-experiment analysis. ROBOT_RUN_STATS tracks per-robot performance metrics. RUN_WORKLOAD_TASKS records individual task assignments and completion status.

JDBC/SSL Communication Path (Port 6543)

The communication path provides bidirectional data exchange between the client application and the cloud database using JDBC over SSL on port 6543. The Database Client module initiates connections through the PostgreSQL JDBC Driver, which establishes secure encrypted communication with the Supabase PostgreSQL instance. Outbound traffic consists of SQL queries, insert statements for new experiment results, and update operations. Inbound traffic carries query result sets, write confirmations, and error responses back to the application. Port 6543 is Supabase's connection pooler port, which manages multiple client connections efficiently. This two-way communication enables the application to save experiment results, retrieve historical run data, and access shared maps stored by team members.

3. Definitions

3.1 Branding

3.1.1 Visual Identity & Feel

The OpenRobotics Warehouse Simulator is designed as a research and engineering tool, so we decided that its branding should emphasize clarity, neutrality, and technical precision, allowing users to focus on system behavior, experimental parameters, and performance metrics without visual distraction.

The interface avoids decorative elements, heavy gradients, playful iconography, and saturated color usage. We aim to prioritize structured layouts and a restrained visual hierarchy.

3.1.2 Logo

The project logo visually represents the system's core themes of **robotics, sensing, coordination, and automation**.

The logo is composed of:

1. A central circular element – perception, awareness, and monitoring.
2. A mechanical gear – machinery, robotics, and automated systems.
3. A blueprint - design, documentation, and knowledge

The logo is intentionally simple and geometric, which enables easy scaling and clear reproduction. Its design allows us to integrate naturally into a neutral, research-oriented interface without overpowering other UI elements.

Good

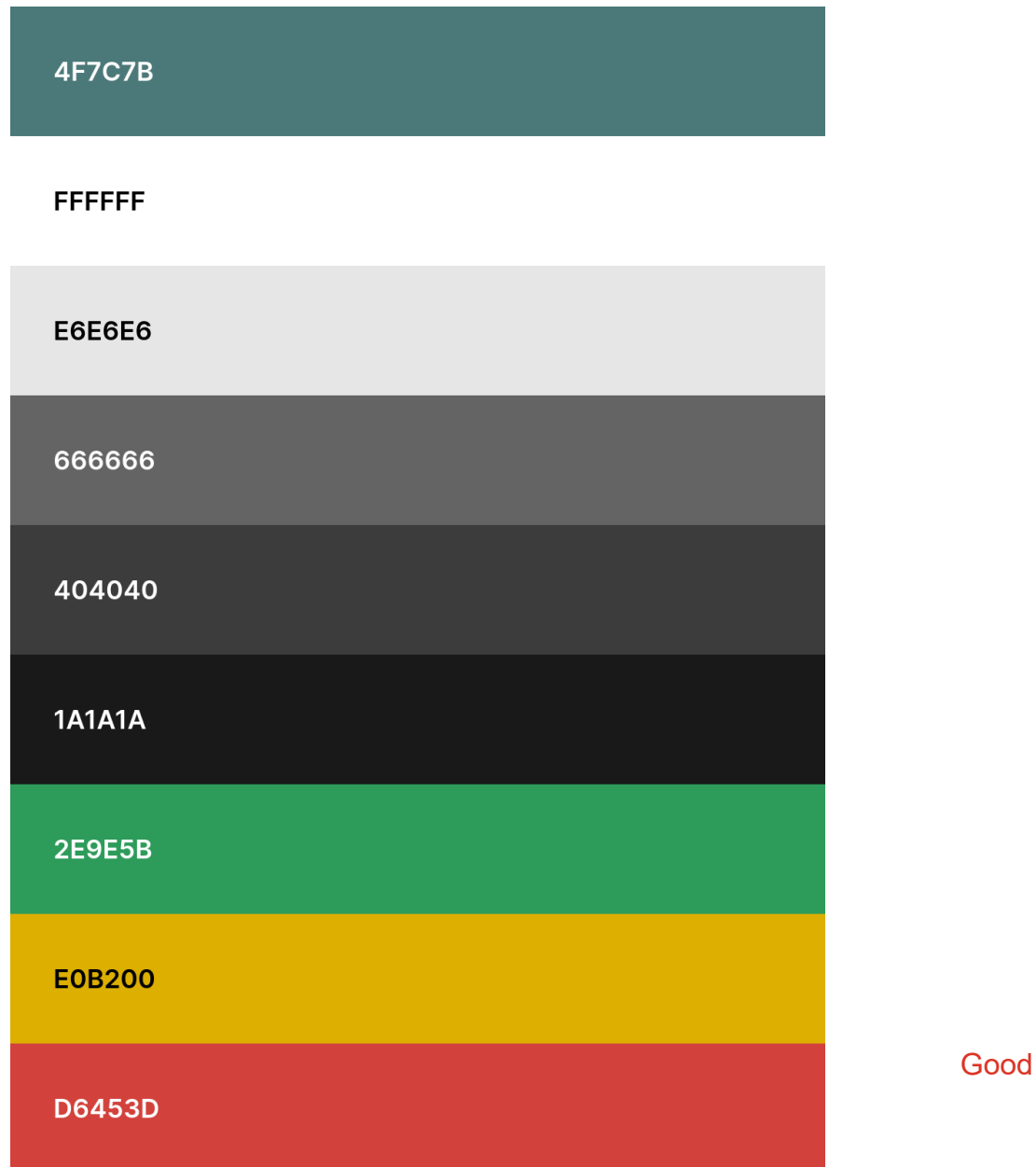
Figure 3.1.2 A: Logo in a light background



Figure 3.1.2 B: Logo in a dark background



3.1.3 Color Palette



Pine Blue – #4F7C7B

Used as the primary brand accent to highlight active UI elements, selections, and key interactive components.

Pure White – #FFFFFF

Used for main backgrounds and surfaces to keep the interface clean, open, and readable.

Light Gray – #E6E6E6

Used for secondary backgrounds and panels to subtly separate content sections.

Medium Gray – #666666

Used for borders, dividers, and secondary UI structure to define layout without visual noise.

Charcoal Gray – #404040

Used for secondary text, icons, and inactive elements to maintain hierarchy without high contrast.

Carbon Black – #1A1A1A

Used for primary text, core icons, and high-contrast elements requiring maximum legibility.

Success Green – #2E9E5B

Used to indicate successful actions, completed tasks, and valid system states.

Warning Yellow – #E0B200

Used to signal caution states such as congestion, near-misses, or performance risks.

Error Red – #D6453D

Used to represent collisions, failures, invalid actions, and critical system errors.

Note: Color palette is subject to change depending on limitations of UI Framework

3.2 Required Technologies

3.2.1 Devices and Hardware Requirements

The simulator is designed to run on standard office laptops and desktop computers. No specialized hardware is required.

Minimum recommended device specifications:

- CPU: Dual-core processor or higher
- RAM: 4 GB minimum
- Storage: 500 MB free disk space
- Display: 1280×800 resolution or higher
- Input: Keyboard and mouse or trackpad

3.2.2 Programming Languages, Libraries, and Frameworks

The system will be implemented primarily using the following technologies:

- **Programming Language:** Java This is hanging, move to next line.

- **UI Framework:** JavaFX
- **Database:** PostgreSQL
- **Data Serialization:** JSON
- **Build System:** Maven
- **Version Control:** Git (GitHub)

Supporting libraries may be used for charting, database connectivity, a more aesthetically pleasing UI, etc.

3.2.3 Runtime Environments

The simulator is a standalone desktop application.

Runtime characteristics:

- Runs locally on the user's machine
- No browser dependency
- No cloud deployment required
- No continuous internet connection required during normal operation **Really?! How?**

The application will be distributed as a packaged JAR file.

3.2.4 Compute, Network, and Server Requirements

All simulation computation is performed locally on the user's machine. The application connects to a Supabase-hosted PostgreSQL database for persistent storage of experiment results, historical run data, and shared maps.

If data is downloaded/uploaded you need to say so here and when that happens.

Network requirements:

- **Required for database operations:** The application communicates with the Supabase cloud server over JDBC/SSL on port 6543 to save and retrieve experiment data.
- **Required during development:** Library dependencies are fetched via network during build.
- **Not required for simulation execution:** Once a simulation is configured and started, no network access is needed until results are saved.

The cloud database was selected to enable all team members to access shared experiment data without hosting a dedicated server. If network connectivity is unavailable, the application can run simulations but cannot persist results to the shared database.

3.3 Project and System Constraints

3.3.1 Budget Constraints

The project has a zero budget.

Constraints:

- Only free and open-source software may be used.
- No paid APIs, cloud services, or licensed tools are permitted.
- Deployment must not incur hosting or infrastructure costs.

3.3.2 Equipment Constraints

All development and testing will be performed using team members' personal laptops.

Constraints include:

- No access to robotics hardware
- No access to warehouse equipment
- No access to physical sensors
- No access to high-performance computing clusters

In short, all experimentation must occur in a simulated environment.

3.3.3 Technical Constraints

The following technical constraints shape the system design:

- The application must be a standalone desktop application.
- The system must run offline after installation.
- The simulation must be deterministic and reproducible.
- All experiments must be configurable via JSON.
- All metrics must be automatically logged.
- All robot hardware parameters must be identical.
- Robots must not self-discover tasks (centralized dispatcher only).
- Collision resolution must be handled by the engine, not by robots.

3.3.4 Scope Constraints

To ensure that we fit a one-semester timeline, the following constraints apply:

- Only grid-based environments are supported.
- Only one robot hardware model is supported.
- Only the defined navigation strategies are supported.
- Only the defined coordination policies are supported.

Good

- No real-time networking between machines is supported.
- No physical robot integration is supported.

3.4 Client Expectations

3.4.1 Must-Have Features

The following features are mandatory for system acceptance:

- Configurable number of robots
- Configurable navigation algorithms
- Configurable coordination policies
- Grid-based map visualization
- Real-time simulation playback
- Pause, resume, stop controls
- Speed control
- Collision detection and logging
- Deadlock detection and recovery
- Task assignment and execution
- Energy modeling and battery tracking
- Metrics calculation
- Results dashboard
- Persistent storage of experiment results
- Configuration save/load functionality

3.4.2 Wants / Nice-to-Haves

The following features are desirable but not required for acceptance:

- Custom map editor
- Advanced heatmap overlays
- Detailed robot inspection panels
- Animation smoothing
- Dark mode
- Advanced filtering of historical runs
- Exportable reports
- Priority-based tasks
- Time-constrained tasks

3.4.3 Assumptions

The following assumptions are made:

- Users have basic familiarity with simulation tools. Hanging, again.

- Users understand the concept of algorithms and metrics.
- Users are comfortable configuring experiments.
- Users are primarily researchers or students.
- The simulator is not used for real-time operational decisions.

3.4.4 Success and Failure Criteria

Success Criteria

The system is considered successful if:

I think you have said this same thing a few times already. This is not good.

- It runs without crashes under normal usage.
- It completes simulations deterministically.
- It logs all required metrics.
- It allows meaningful comparison of algorithm configurations.
- It produces consistent and reproducible results.
- It is usable without external assistance.

Failure Criteria

The system is considered unsuccessful if:

- It frequently crashes or freezes.
- Results are inconsistent across identical runs.
- Metrics are incorrect or missing.
- Collisions are not handled safely.
- The UI is confusing or misleading.
- The system cannot complete basic workloads.

3.4.5 Explicit Scope Statements

Will Do

okay

- Simulate warehouse robots
- Support multiple navigation algorithms
- Support multiple coordination policies
- Visualize robot movement
- Log and analyze performance metrics
- Support experimental configuration

Will Not Do

- Control real robots
- Integrate with physical warehouses
- Support arbitrary algorithms

- Perform real-time optimization
- Predict real-world financial outcomes
- Provide real-time networked multi-user sessions

3.5 Impact, Ethics, and Performance Statements

3.5.1 Impact Statement

This simulator provides a controlled and free environment for exploring the behavior of multi-robot systems. It enables students and researchers to experiment with coordination strategies without the risks, costs, or safety concerns.

The tool can contribute to improved warehouse system design by highlighting the trade-offs between throughput, congestion, and energy efficiency.

3.5.2 Ethical Considerations

The simulator does not collect personal data and does not interact with real-world systems.

Ethical considerations include:

- Avoiding misrepresentation of results as real-world guarantees
- Preventing misuse as a real-time decision-making tool
- Clearly communicating that the tool is a simulation, not a prediction engine

All results are contextual and must be interpreted as experimental outcomes, not operational advice.

3.5.3 Performance Statements

The system is designed to:

- Support a couple of robots simultaneously "couple" is not precise enough.
- Render real-time movement smoothly
- Execute simulations deterministically
- Store large experiment histories
- Handle long-running simulations

Performance targets include:

- Stable frame rates during simulation
- No memory leaks
- No unbounded CPU growth
- Predictable runtime behavior

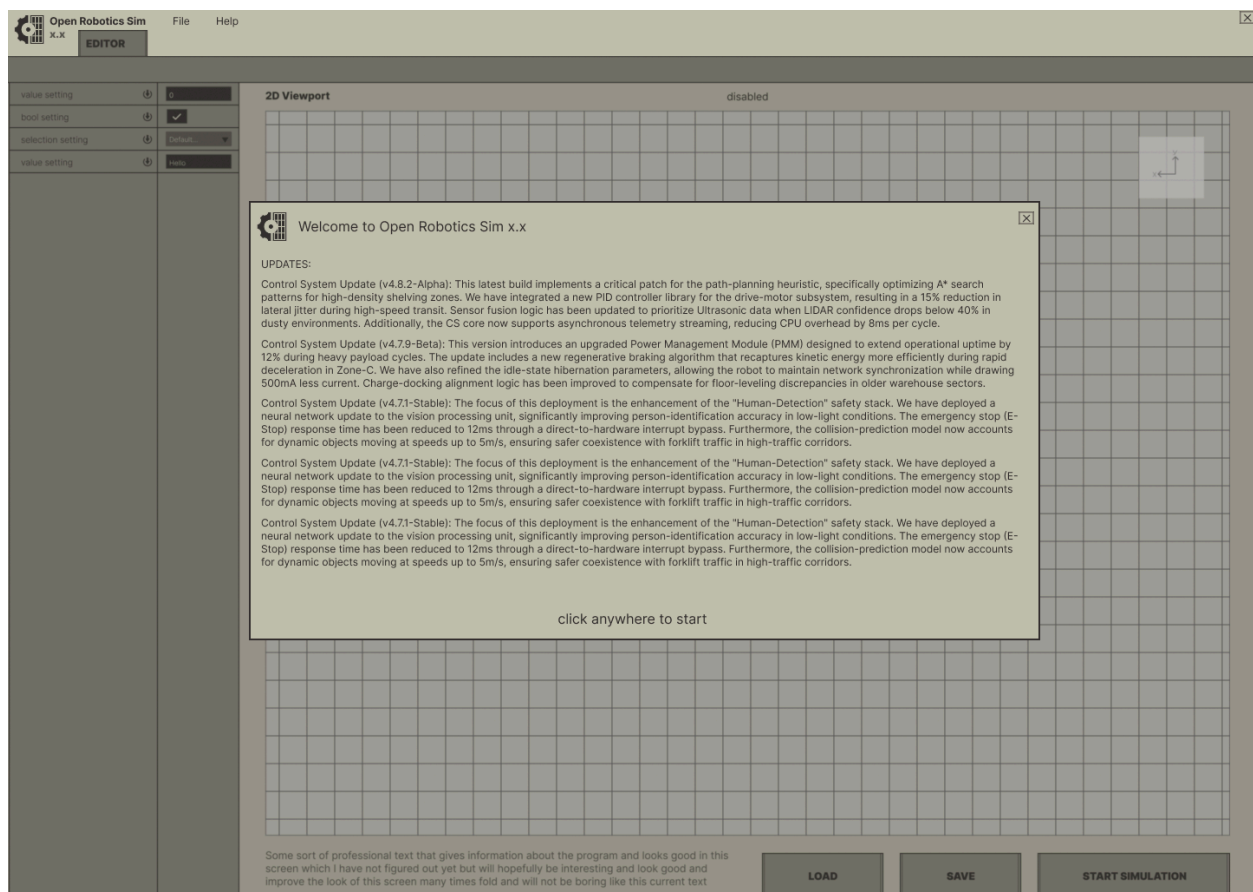
4. Screen Layouts

Note that the application is meant to be used in fullscreen on standard displays and every mockup shown below applies to fullscreen resolution only.

4.1 Screen Mockups

4.1.1 Welcome Screen

Figure 4.1.1 : Welcome Screen Mockup

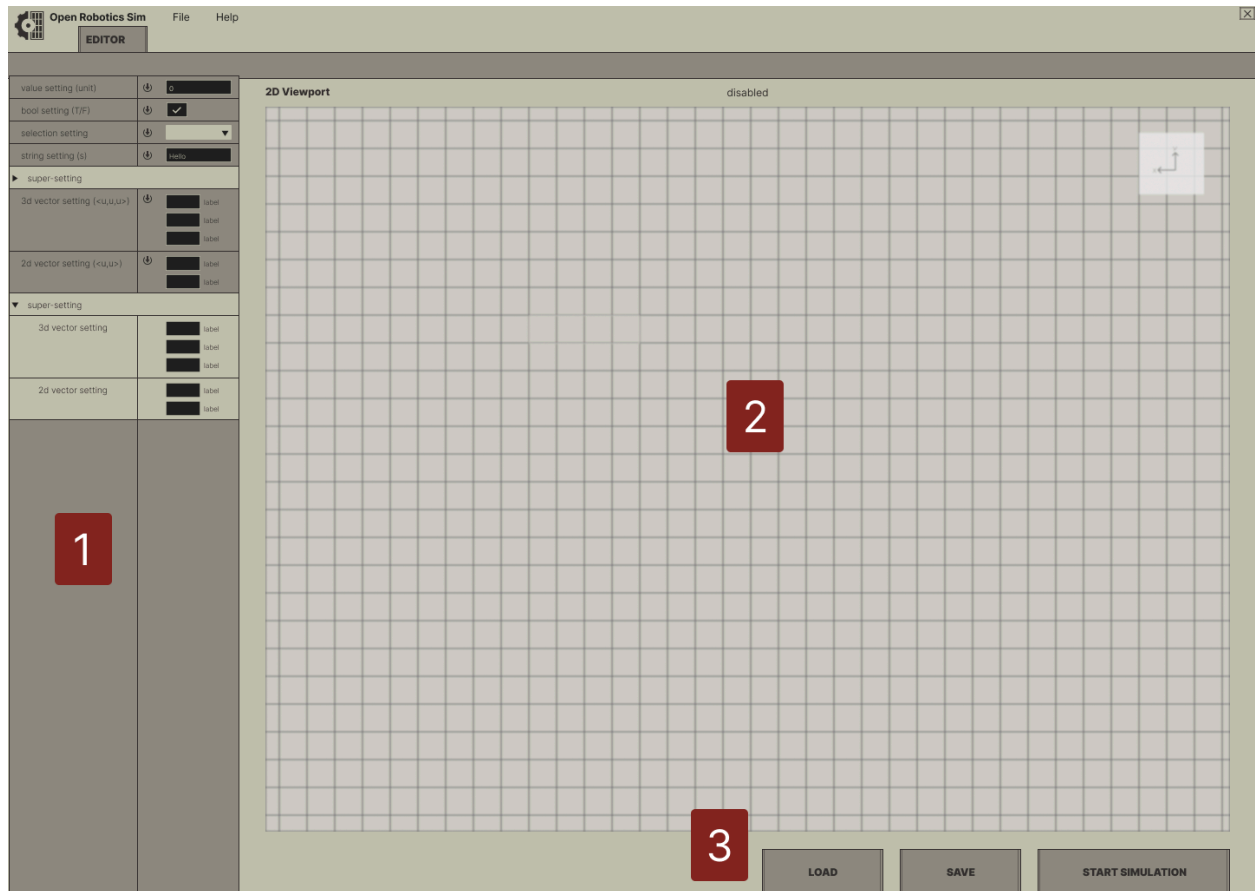


On this screen, the user can view the update logs as well as click anywhere to begin using the application.

okay

4.1.2 Setup Screen

Figure 4.1.2 : Setup Screen Mockup



1: This part of the setup screen contains all of the configuration settings to fine tune the simulation to the user's needs. The arrow down button inside a circle (left of each input box) is used to revert a parameter to its default value(s).

2: This part of the screen contains a disabled, non-interactive viewport.

3: Users may load a config file or save a file by pressing the respective buttons which opens up the respective specialty screens for setting a directory or selecting a file (see figure 4.2.1). The simulation can also be started using the "Start Simulation" button which takes users to the simulation screen.

good

4.1.3 Simulation Screen

Figure 4.1.3 A: Simulation Screen Mockup

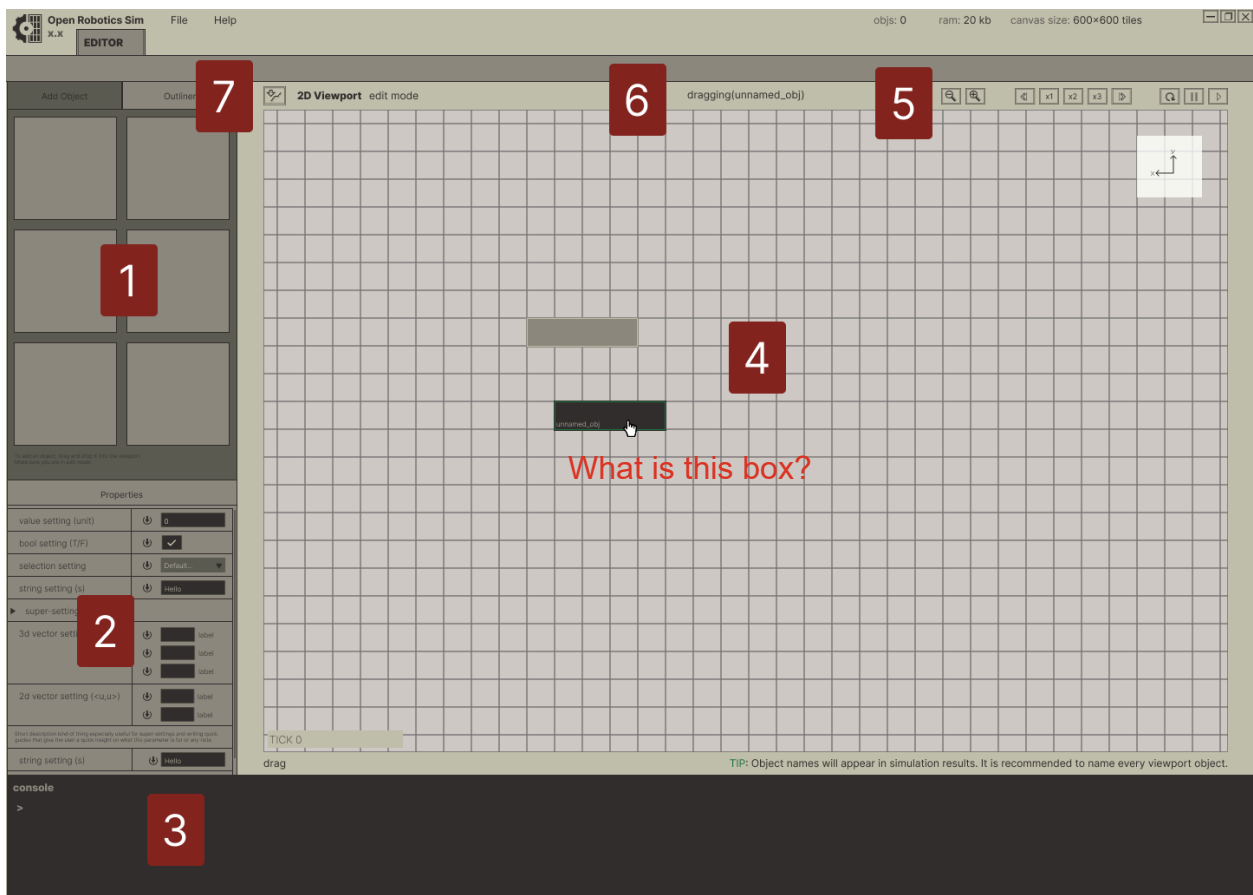
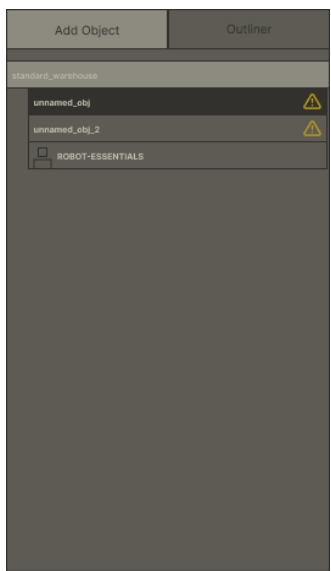


Figure 4.1.3 B: Outliner Window Mockup



1: This window is divided into two parts. The first is the “Add Object” tab which lets you drag and drop objects into the viewport. Two left clicks on the square boxes (which will contain icons for objects) will also open up the specific object's description's specialty screen which includes a brief overview of its properties and use-cases (see the section on specialty screens). The second is the “Outliner Tab” which gives an overview of the existing objects in the viewport and facilitates their selection.

2: This part shows the properties of a specific selected object in the viewport. They can be modified on the right side by pressing on the various designated input boxes.

3: The console here can be interacted with as with any console, i.e., first left-clicking it to begin typing and then typing predefined program commands. The console also includes more detailed error logs.

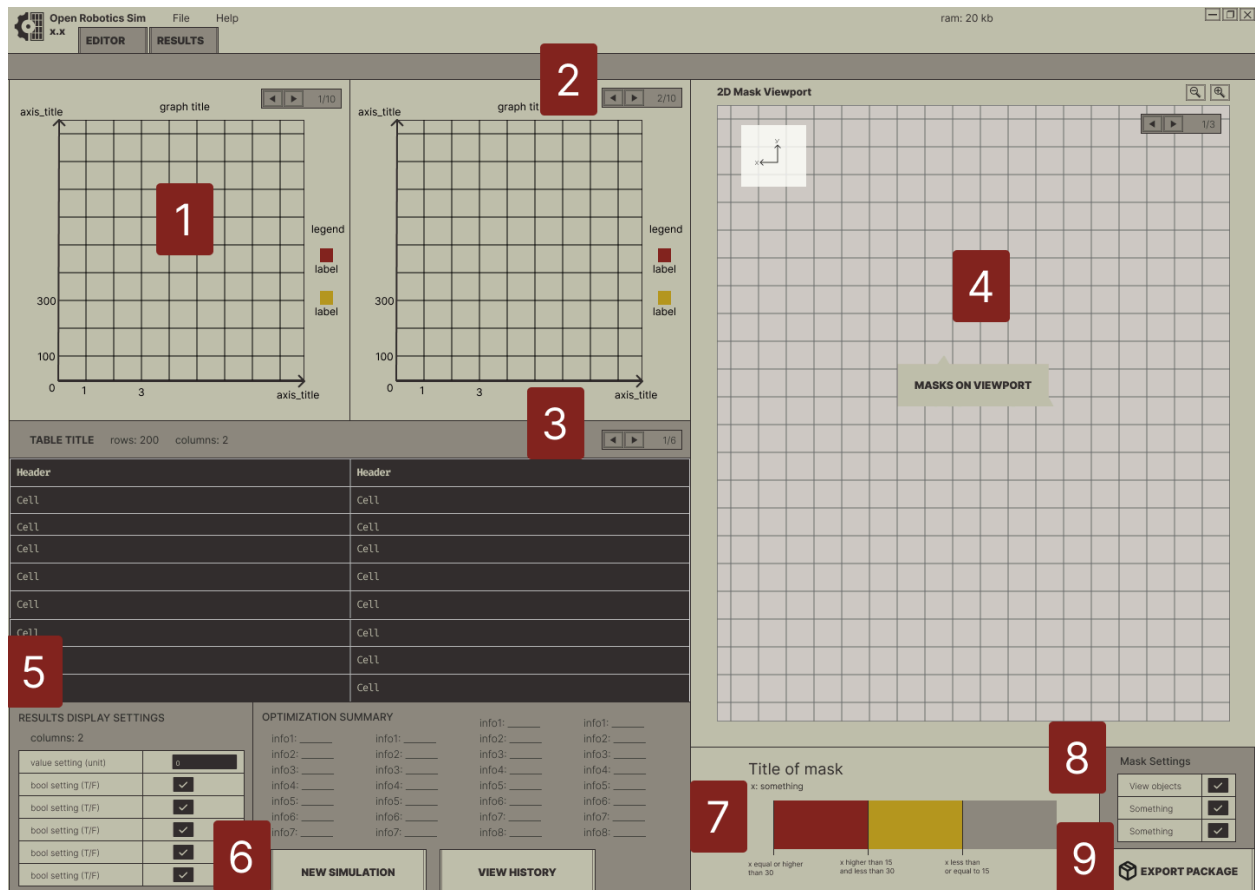
4: The viewport can be interacted with in various ways. Holding the right click enters the user into “movement mode” (shown right of the “2d Viewport label”). To move around the 2d space, the user then presses W, A, S, D keys for respectively top, left, bottom and right movement. Alternatively, the user may hold the left mouse click and move the mouse towards the desired direction (in both cases, the right click of the mouse is still held). To move an object, one simply left-clicks the object (which then highlights it) and moves it in “edit mode”. Other viewport interactions include using the scroll wheel to zoom in and out.

5: These buttons enable the user to manually zoom in/out, run, pause or restart the simulation and even change the simulation's speed.

6: This label shows the specific function working on the viewport at any given time and the object it affects.

4.1.3 Results Screen

Figure 4.1.3: Results Screen Mockup



1: This graph screen cannot be interacted with.

2: The two buttons are used for navigating through a number of paginated graphs

okay

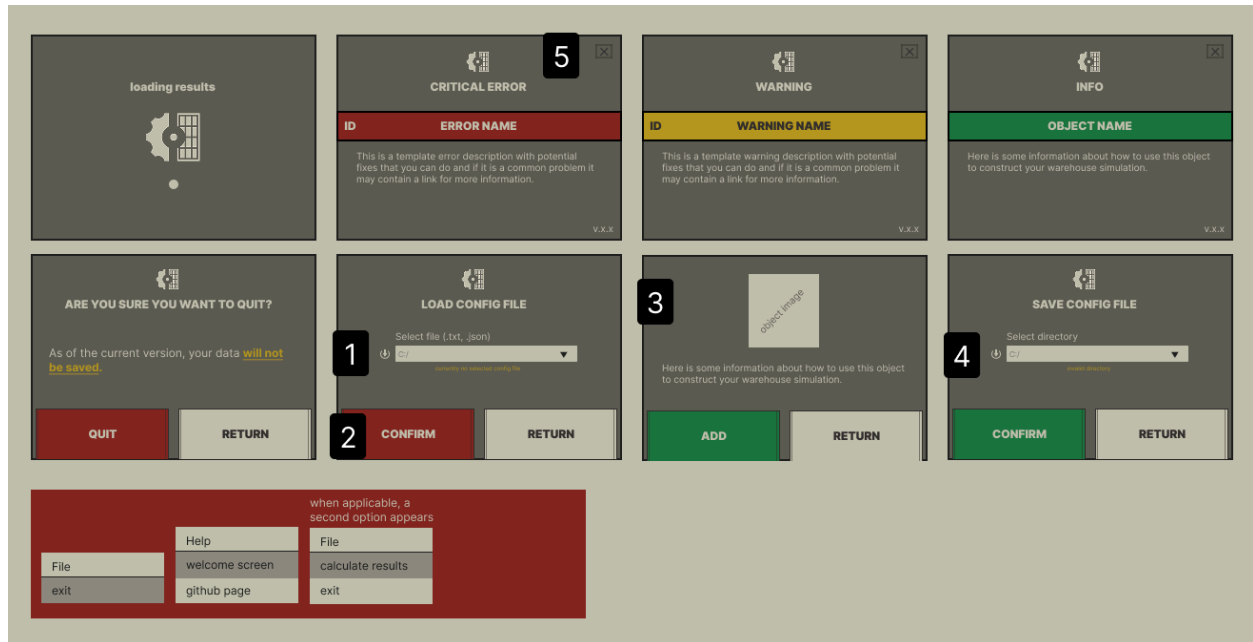
3: The two buttons are used for navigating through a number of paginated tables. The table itself includes a scroll wheel which only appears when hovering over the table. The user can also use their scroll wheel to view lower rows.

4: The viewport here is used to display 2d masks such as the frequency of travel on each grid square by the robots.

5: The results display settings are used for mainly customization of the results screen. The input boxes work the same way that the object properties' window works.

4.2 Specialty Screens

Figure 4.2 : Specialty Screens



1: Pressing the down arrow displays a selection list of cached files alongside a last option to manually look up and add a file using the user's OS-specific file explorer. The white down arrow button reverts to the default config file.

2: These lower buttons indicate the last step of specialty screen interactions.

3: This screen appears when double clicking an object in the "Add Object" window of the simulation screen. The Add button will add the object at the origin of the viewport.

okay

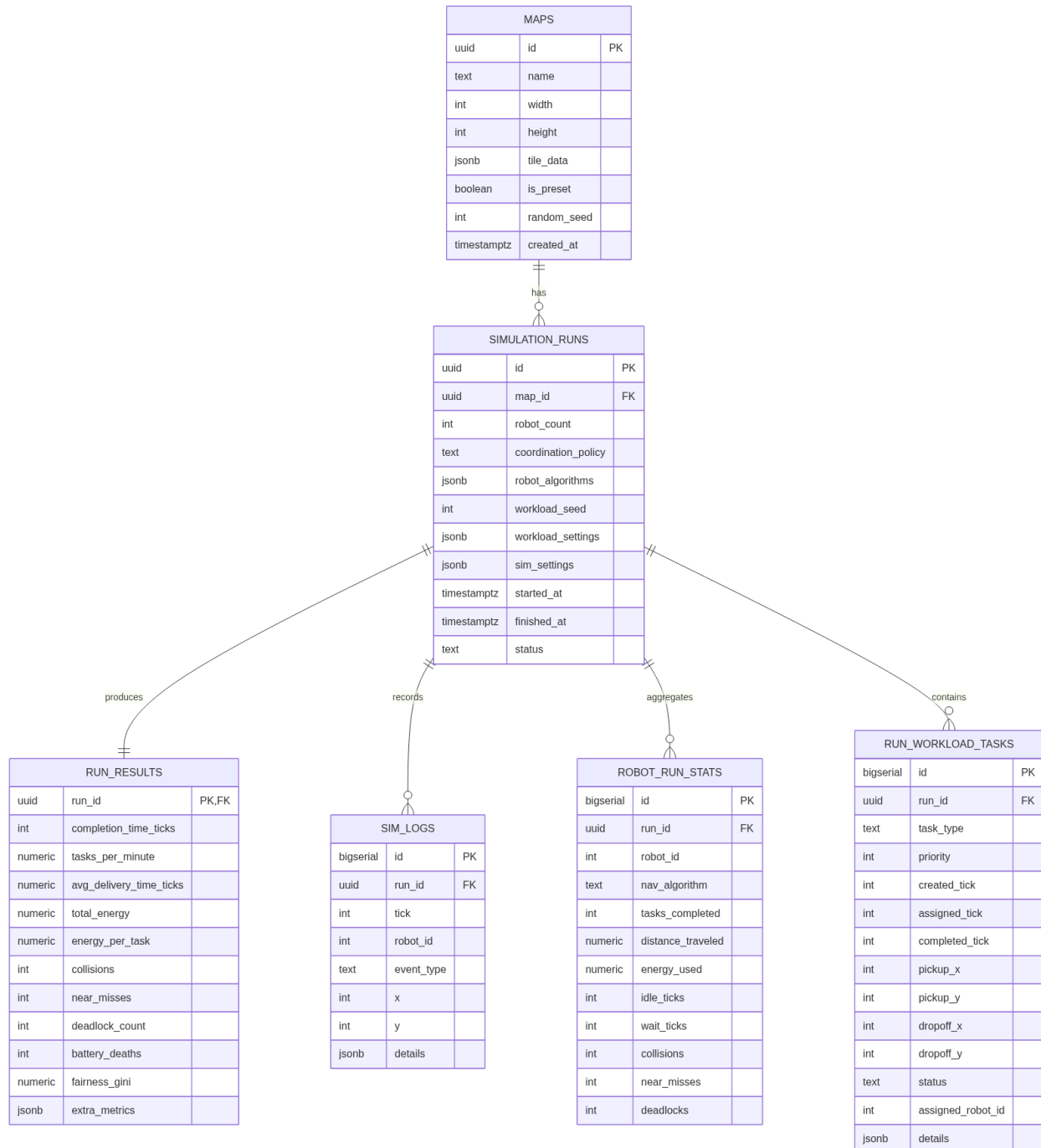
4: Pressing the down arrow displays a selection of cached directories alongside a last option to manually search and provide a directory using the user's OS-specific file explorer. The white down arrow button reverts to the default config file.

5. Sections & Queries

5.1 Defining File Structures

Need text between each numbered section, eg. 5.1 and 5.1.1.

5.1.1 Database Diagram



5.1.2 Database table structures

Don't just provide tables and diagrams without at least one sentence of a short paragraph.

5.1.2.1 Maps Table

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	UUID	36 chars	Primary key	Not null, auto-generated
name	String	100 chars	Index (optional)	Not null
width	Int	4 digits	–	> 0
height	Int	4 digits	–	> 0
tile_data	JSONB	–	–	Not null, valid map JSON structure
is_preset	Boolean	1	–	Default false
random_seed	Int	10 digits	–	Nullable (null for presets)
created_at	Timestamp	–	Sort DESC	Default now

5.1.2.2 Simulation_Runs Table

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	UUID	36 chars	Primary key	Not null, auto-generated
map_id	UUID	36 chars	Foreign key → maps(id)	Not null
robot_count	Int	4 digits	–	≥ 1
coordination_policy	String	30 chars	–	Must be TRAFFIC_RULES or RESERVATION_K
robot_algorithms	JSONB	–	–	Not null (robotId → algorithm mapping)
workload_seed	Int	10 digits	–	Not null
workload_settings	JSONB	–	–	Not null
sim_settings	JSONB	–	–	Not null

Field Name	Type	Length	Key, Primary, Sort	Constraints
started_at	Timestamp	–	Sort DESC	Default now
finished_at	Timestamp	–	–	Nullable
status	String	15 chars	Index	Must be one of : CREATED, RUNNING, COMPLETED, FAILED, CANCELLED

5.1.2.3 Run_Results Table

Field Name	Type	Length	Key, Primary, Sort	Constraints
run_id	UUID	36 chars	Primary key + FK → simulation_runs(id)	Not null, cascade delete
completion_time_ticks	Int	10 digits	–	Nullable if failed/cancelled
tasks_per_minute	Decimal	–	–	≥ 0
avg_delivery_time_ticks	Decimal	–	–	≥ 0
total_energy	Decimal	–	–	≥ 0
energy_per_task	Decimal	–	–	≥ 0
collisions	Int	10 digits	–	Default 0, ≥ 0
near_misses	Int	10 digits	–	Default 0, ≥ 0
deadlock_count	Int	10 digits	–	Default 0, ≥ 0
battery_deaths	Int	10 digits	–	Default 0, ≥ 0
fairness_gini	Decimal	–	–	Typically 0..1 (enforce in app layer)
extra_metrics	JSONB	–	–	Default {}

5.1.2.4 Sim_Logs Table

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	BigSerial	–	Primary key	Auto-increment
run_id	UUID	36 chars	Foreign key → simulation_runs(id)	Not null
tick	Int	10 digits	Sort ASC	≥ 0
robot_id	Int	10 digits	Index (optional)	Nullable for global events
event_type	String	30 chars	Index	Not null (enum enforced in app)
x	Int	10 digits	–	Nullable
y	Int	10 digits	–	Nullable
details	JSONB	–	–	Default <code>{}</code>

5.1.2.5 Robot_Run_Stats Table

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	BigSerial	–	Primary key	Auto-increment
run_id	UUID	36 chars	Foreign key → simulation_runs(id)	Not null
robot_id	Int	10 digits	Unique per run	Unique(run_id, robot_id)
nav_algorithm	String	15 chars	–	Must be one of the defined algorithms
tasks_completed	Int	10 digits	–	Default 0
distance_traveled	Decimal	–	–	≥ 0
energy_used	Decimal	–	–	≥ 0
idle_ticks	Int	10 digits	–	≥ 0
wait_ticks	Int	10 digits	–	≥ 0

Field Name	Type	Length	Key, Primary, Sort	Constraints
collisions	Int	10 digits	—	≥ 0
near_misses	Int	10 digits	—	≥ 0
deadlocks	Int	10 digits	—	≥ 0

5.1.2.6 Run_Workload_Tasks Table

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	BigSerial	—	Primary key	Auto-increment
run_id	UUID	36 chars	Foreign key → simulation_runs(id)	Not null
task_type	String	15 chars	—	PICK_DELIVER or REPLENISH
priority	Int	4 digits	—	Default 0
created_tick	Int	10 digits	Sort ASC	≥ 0
assigned_tick	Int	10 digits	—	Nullable
completed_tick	Int	10 digits	—	Nullable
pickup_x	Int	10 digits	—	Not null
pickup_y	Int	10 digits	—	Not null
dropoff_x	Int	10 digits	—	Not null
dropoff_y	Int	10 digits	—	Not null
status	String	12 chars	Index	CREATED/ASSIGNED/COMPLETED/CANCELLED/FAILED
assigned_robot_id	Int	10 digits	—	Nullable
details	JSONB	—	—	Default {}

5.1.3 Formatted text files

5.1.3.1 CSV Export: results_runs.csv

CSV Header: run_id, map_name, robot_count, coordination_policy, workload_seed, status, completion_time_ticks, tasks_per_minute, avg_delivery_time_ticks, total_energy, energy_per_task, collisions, near_misses, deadlock_count, battery_deaths, fairness_gini

Constraints / Notes:

- run_id is UUID (string)
- Numeric fields must be non-negative
- status $\in$ {CREATED,RUNNING,COMPLETED,FAILED,CANCELLED}
- One row per simulation run

5.1.3.2 CSV Export: results_robots.csv

CSV Header: run_id, robot_id, nav_algorithm, tasks_completed, distance_traveled, energy_used, idle_ticks, wait_ticks, collisions, near_misses, deadlocks

Constraints / Notes:

- nav_algorithm $\in$ {defined algorithms structures}
- robot_id is integer ≥ 0
- One row per robot per run (unique: run_id + robot_id)

5.1.3.3 JSON File: map.json

JSON Structure (header-style):

```
{
  "version": 1,
  "name": "string",
  "width": "integer",
  "height": "integer",
  "randomSeed": "integer|null",
  "tiles": "2D array of strings",
  "pointsOfInterest": {
    "packingStations": "array",
    "docks": "array",
    "chargers": "array"
  }
}
```

okay

5.1.3.4 JSON File : run_config.json

JSON Structure (header-style):

```
{
  "version": 1,
  "runName": "string",
  "mapRef": {
    "type": "preset|db|file",
    "name": "string (if preset)",
    "mapId": "uuid (if db)",
    "path": "string (if file)"
  },
  "coordinationPolicy": {
    "type": "TRAFFIC_RULES|RESERVATION_K",
    "k": "integer (required only if RESERVATION_K)"
  },
  "robots": [
    { "robotId": "integer", "navAlgorithm": "GREEDY|BUG|RTA_STAR" }
  ],
  "workload": {
    "seed": "integer",
    "mode": "SPAWN_RATE|FIXED_LIST",
    "spawnRatePerMin": "number (if SPAWN_RATE)",
    "maxTasks": "integer (if SPAWN_RATE)"
  },
  "simulation": {
    "maxTicks": "integer",
    "tickMs": "integer"
  }
}
```

User-created configuration files are saved to `~/open-robotics/configs/` and use the naming convention `{runName}.json`. The `mapRef.path` field, when used with type: "file", should contain an absolute path or a path relative to `~/open-robotics/maps/`. Preset maps (bundled in the JAR) are referenced by name only and do not use the path field.

5.1.4 Defined Operations

5.1.4.1 Maps Table Queries

SQL	Motivation
<code>SELECT * FROM maps;</code>	List all maps

SQL	Motivation
<pre>SELECT id, name, width, height FROM maps WHERE is_preset = TRUE ORDER BY name;</pre>	List preset maps for UI dropdown
<pre>SELECT * FROM maps WHERE random_seed = :seed;</pre>	Reproduce / locate a specific randomized map

Good

5.1.4.2 Simulation_Runs Table Queries

SQL	Motivation
<pre>SELECT * FROM simulation_runs ORDER BY started_at DESC LIMIT 20;</pre>	Recent runs dashboard
<pre>SELECT * FROM simulation_runs WHERE map_id = :map_id AND robot_count = :n;</pre>	Filter runs by scenario parameters
<pre>SELECT id, status FROM simulation_runs WHERE status IN ('RUNNING', 'CREATED');</pre>	Find active runs to control from UI

5.1.4.3 Run_Results Table Queries

SQL	Motivation
<pre>SELECT * FROM run_results WHERE run_id = :run_id;</pre>	Show final results page for one run
<pre>SELECT r.robot_count, AVG(rr.total_energy) AS avg_energy FROM simulation_runs r JOIN run_results rr ON rr.run_id = r.id WHERE r.map_id = :map_id AND r.status = 'COMPLETED' GROUP BY r.robot_count ORDER BY r.robot_count;</pre>	Energy vs robot count curve
<pre>SELECT r.robot_count, AVG(rr.collisions) AS avg_collisions FROM simulation_runs r JOIN run_results rr ON rr.run_id = r.id WHERE r.coordination_policy = :policy AND r.status = 'COMPLETED' GROUP BY r.robot_count ORDER BY r.robot_count;</pre>	Congestion trend under a policy

5.1.4.4 Sim_Logs Table Queries

SQL	Motivation
<pre>SELECT * FROM sim_logs WHERE run_id = :run_id ORDER BY tick ASC;</pre>	Full replay / debug timeline
<pre>SELECT tick, robot_id, x, y, details FROM sim_logs WHERE run_id = :run_id AND event_type = 'COLLISION' ORDER BY tick;</pre>	Collision analysis
<pre>SELECT x, y, COUNT(*) AS hits FROM sim_logs WHERE run_id = :run_id AND event_type = 'MOVE_EXECUTED' GROUP BY x, y ORDER BY hits DESC LIMIT 20;</pre>	Heatmap hotspot extraction

5.1.4.5 Robot_Run_Stats Table Queries

SQL	Motivation
<pre>SELECT * FROM robot_run_stats WHERE run_id = :run_id ORDER BY tasks_completed DESC;</pre>	Fairness/workload distribution
<pre>SELECT nav_algorithm, AVG(energy_used) AS avg_energy FROM robot_run_stats WHERE run_id = :run_id GROUP BY nav_algorithm ORDER BY avg_energy;</pre>	Compare algorithm energy use
<pre>SELECT robot_id, wait_ticks FROM robot_run_stats WHERE run_id = :run_id ORDER BY wait_ticks DESC;</pre>	Find robots most affected by congestion

5.1.4.6 Run_Workload_Tasks Table Queries

SQL	Motivation
<pre>SELECT COUNT(*) FROM run_workload_tasks WHERE run_id = :run_id AND status = 'COMPLETED';</pre>	Total completed tasks

SQL	Motivation
<pre>SELECT (completed_tick - created_tick) AS delivery_ticks FROM run_workload_tasks WHERE run_id = :run_id AND status = 'COMPLETED';</pre>	Delivery-time dataset for histogram
<pre>SELECT assigned_robot_id, COUNT(*) AS completed FROM run_workload_tasks WHERE run_id = :run_id AND status = 'COMPLETED' GROUP BY assigned_robot_id ORDER BY completed DESC;</pre>	Fairness by completed tasks

5.2 Remote API definitions

5.2.1 Global API Rules (Applies to All Endpoints)

Base URL

- /api/v1

Missing API signatures

Content Type

- Requests that include a body must be application/json
- Responses are structured as JSON-like objects (described below using key:value bullets, not JSON formatting)

Authentication (MVP)

- No authentication required (local project use)

Input Validation Rules

- All required fields must be present.
- Enum fields are case-sensitive and must match allowed values exactly:
 - coordinationPolicy.type must be one of the coordination policies types.
 - robots[].navAlgorithm must be one of the defined algorithm methods.
 - run.status must be one of: CREATED, RUNNING, COMPLETED, FAILED, CANCELLED
 - log.eventType must be one of:
 - TASK_CREATED, TASK_ASSIGNED, TASK_COMPLETED
 - MOVE_INTENT, MOVE_EXECUTED
 - COLLISION, NEAR_MISS
 - CHARGE_START, CHARGE_END
 - DEADLOCK_DETECTED, DEADLOCK_RESOLVED
 - BATTERY_DEATH

- RUN_STARTED, RUN_COMPLETED, RUN_FAILED
- Numeric fields must be valid numbers (not text) and satisfy constraints:
 - Any count or tick value must be ≥ 0
 - simulation.maxTicks must be > 0
 - simulation.tickMs must be > 0
 - If coordinationPolicy.type = RESERVATION_K, then coordinationPolicy.k must exist and be ≥ 1
- IDs must be valid UUID strings:
 - mapId must refer to an existing map
 - runId must refer to an existing run
- Coordinates must be within map bounds:
 - $0 \leq x < \text{width}$
 - $0 \leq y < \text{height}$

Standard Error Return Structure (used by all endpoints)

- error:
 - code: one of VALIDATION_ERROR, NOT_FOUND, CONFLICT, SERVER_ERROR
 - message: human readable error message
 - fields (optional): dictionary of fieldName $\rightarrow$ errorDescription

Pagination Rules (List endpoints)

- Optional query params:
 - limit default 50, maximum 200
 - offset default 0
- Return includes:
 - limit, offset, total

5.2.2 Maps API

5.2.2.1 API Call Signatures

- GET /maps/list
- GET /maps/get/{mapId} okay
- GET /maps/search?name={substring}&presetOnly={true|false}
- POST /maps/create

5.2.2.2 GET /maps/list

Description and Usage Rules

- Lists all available maps (preset and randomized).
- Used by GUI dropdown to select a map.
- Pagination supported using limit and offset.

Input Validation Requirements

- limit must be an integer in range 1..200 (if provided)
- offset must be an integer ≥ 0 (if provided)

Return Format Structure (JSON-like)

- listing: array of map summary objects
 - mapId: UUID string
 - name: string
 - width: integer
 - height: integer
 - isPreset: boolean
 - randomSeed: integer or null
- limit: integer
- offset: integer
- total: integer

Example Response

- listing:
 - item 1:
 - mapId: b7c7f0c8-6e25-4c3c-9d08-9f2a2acb6f6e
 - name: narrow_aisles
 - width: 40
 - height: 25
 - isPreset: true
 - randomSeed: null
- limit: 50
- offset: 0
- total: 1

5.2.2.3 GET /maps/get/{mapId}

Description and Usage Rules

- Returns full map definition including tile grid and points of interest.
- Used by engine to load the map for a run, and by GUI to render the map.

Input Validation Requirements

- mapId must be a valid UUID string
- mapId must exist; otherwise return NOT_FOUND

Return Format Structure (JSON-like)

- map:
 - mapId: UUID string
 - name: string
 - width: integer
 - height: integer
 - isPreset: boolean
 - randomSeed: integer or null
 - tileData: map definition object (stored format used by the engine)

- version: integer
- tiles: 2D array (height rows × width columns)
- pointsOfInterest:
 - packingStations: array of objects (id, x, y)
 - docks: array of objects (id, x, y)
 - chargers: array of objects (id, x, y)

Example Response

- map:
 - mapId: b7c7f0c8-6e25-4c3c-9d08-9f2a2acb6f6e
 - name: narrow_aisles
 - width: 40
 - height: 25
 - isPreset: true
 - randomSeed: null
 - tileData:
 - version: 1
 - tiles: (grid omitted in example)
 - pointsOfInterest:
 - packingStations:
 - item 1: id=PS1, x=2, y=12
 - docks:
 - item 1: id=D1, x=18, y=12
 - chargers:
 - item 1: id=C1, x=1, y=1

5.2.2.4 GET /maps/search?name={substring}

Description and Usage Rules

- Searches maps by name substring.
- Used by GUI search bar or filter panel.

Input Validation Requirements

- name must be a non-empty string (length ≥ 1)

Return Format Structure (JSON-like)

- listing: array of map summary objects
 - mapId, name, width, height, isPreset, randomSeed

Example Response

- listing:
 - item 1:
 - mapId: e14d2b1a-2ad1-4f7b-9f63-3e08e4d8a123
 - name: baseline_small
 - width: 20

- height: 14
- randomSeed: null

5.2.2.5 POST /maps/create

Description and Usage Rules

- Uploads/creates a new map definition.
- Used by randomized map generator and optional custom editor.
- Map is stored in database and becomes available for runs.

Input Validation Requirements

- Required body fields:
 - name (string)
 - width (integer > 0)
 - height (integer > 0)
 - tileData (map definition)
- Must pass map validator rules:
 - tile grid dimensions must match width and height
 - tile values must be valid supported tile types
 - at least 1 charger exists
 - POIs are in bounds
 - packing stations must be reachable from walkable tiles (connectivity)

Return Format Structure (JSON-like)

- created:
 - mapId: UUID string
 - name: string

Example Response

- created:
 - mapId: 5c7f46d1-4a20-4f7b-8e9e-0c8db8a2f2aa
 - name: my_custom_map

5.2.3 Runs API

5.2.3.1 API Call Signatures

- POST /runs/create
- POST /runs/start/{runId}
- POST /runs/stop/{runId}
- GET /runs/status/{runId} Good
- GET /runs/list
- GET /runs/search?mapId={uuid}&robotCount={int}&policy={DEFINED_POLICIES}

5.2.3.2 POST /runs/create

Description and Usage Rules

- Creates a simulation run record from provided configuration.
- Does not start the simulation (use /runs/start/{runId}).
- Used by UI “Create Run” flow and by batch experiment scripts.

Input Validation Requirements

- Required fields in body:
 - runName (string)
 - mapId (UUID string, must exist)
 - coordinationPolicy (object)
 - robots (array, length ≥ 1)
 - workload (object)
 - simulation (object)
- Robots validation:
 - robots[].robotId must be unique
 - robots[].navAlgorithm must be one of GREEDY/BUG/RTA_STAR
- Coordination policy validation:
 - coordinationPolicy.type must be TRAFFIC_RULES or RESERVATION_K
 - if RESERVATION_K then coordinationPolicy.k must exist and be ≥ 1
- Simulation validation:
 - simulation.maxTicks > 0
 - simulation.tickMs > 0
- Workload validation (MVP spawn-rate mode):
 - workload.seed is integer
 - workload.mode is SPAWN_RATE or FIXED_LIST
 - if mode=SPAWN_RATE then spawnRatePerMin ≥ 0 and maxTasks ≥ 0

Return Format Structure (JSON-like)

- created:
 - runId: UUID string
 - status: CREATED

Example Response

- created:
 - runId: 2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc
 - status: CREATED

5.2.3.3 POST /runs/start/{runId}

Description and Usage Rules

- Starts a previously created run.
- Simulation engine begins ticking and logging events.
- Used by UI “Start” button and automated experiment runner.

Input Validation Requirements

- runId must exist
- run must be in status CREATED
- if run status is RUNNING, return CONFLICT

Return Format Structure (JSON-like)

- run:
 - runId: UUID string
 - status: RUNNING

Example Response

- run:
 - runId: 2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc
 - status: RUNNING

5.2.3.4 POST /runs/stop/{runId}

Description and Usage Rules

- Stops a running simulation run.
- Used by UI “Stop” button to end long runs early.

Input Validation Requirements

- runId must exist
- run must be RUNNING; otherwise return CONFLICT

Return Format Structure (JSON-like)

- run:
 - runId: UUID string
 - status: CANCELLED

Example Response

- run:
 - runId: 2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc
 - status: CANCELLED

5.2.3.5 GET /runs/status/{runId}

Description and Usage Rules

- Returns current status and progress of a run.
- Used by UI status panel and progress bar polling.

Input Validation Requirements

- runId must exist

Return Format Structure (JSON-like)

- run:

- runId: UUID string
- status: one of CREATED/RUNNING/COMPLETED/FAILED/CANCELLED
- currentTick: integer ≥ 0
- maxTicks: integer > 0
- progressPct: number in range 0..100

Example Response

- run:
 - runId: 2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc
 - status: RUNNING
 - currentTick: 8120
 - maxTicks: 30000
 - progressPct: 27.07

5.2.3.6 GET /runs/list

Description and Usage Rules

- Lists most recent runs for the dashboard.
- Pagination supported.

Input Validation Requirements

- limit integer 1..200 (optional)
- offset integer ≥ 0 (optional)

Return Format Structure (JSON-like)

- listing: array of run summary objects
 - runId: UUID string
 - runName: string
 - mapId: UUID string
 - robotCount: integer ≥ 0
 - coordinationPolicy: string
 - status: string
 - startedAt: timestamp string
 - finishedAt: timestamp string or null
- limit: integer
- offset: integer
- total: integer

Example Response

- listing:
 - item 1:
 - runId: 2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc
 - runName: baseline_8_robots
 - mapId: b7c7f0c8-6e25-4c3c-9d08-9f2a2acb6f6e
 - robotCount: 8
 - coordinationPolicy: TRAFFIC_RULES

- status: COMPLETED
- startedAt: 2026-01-16T18:05:12Z
- finishedAt: 2026-01-16T18:06:31Z
- limit: 50
- offset: 0
- total: 1

5.2.3.7 GET /runs/search?mapId={uuid}&robotCount={int}&policy={...}

Description and Usage Rules

- Searches runs by parameters for comparative experiments.
- Used by analysis dashboard to filter runs.

Input Validation Requirements

- mapId must be valid UUID (optional)
- robotCount must be integer ≥ 0 (optional)
- policy must be one of the defined policies

Return Format Structure (JSON-like)

- listing: array of run summary objects (same as /runs/list)

Example Response

- listing:
 - item 1:
 - runId: a1111111-2222-3333-4444-555555555555
 - runName: narrow_aisles_12robots
 - mapId: b7c7f0c8-6e25-4c3c-9d08-9f2a2acb6f6e
 - robotCount: 12
 - coordinationPolicy: RESERVATION_K
 - status: COMPLETED
 - startedAt: 2026-01-16T19:00:00Z
 - finishedAt: 2026-01-16T19:04:10Z

5.2.4 Results API

5.2.4.1 API Call Signatures

- GET /results/run/{runId}
- GET /results/robots/{runId}
- GET /results/compare?mapId={uuid}&policy={DEFINED_POLICIES}

5.2.4.2 GET /results/run/{runId}

Description and Usage Rules

- Returns aggregated metrics for a run.
- Used by UI stats dashboard and experiment comparison graphs.

Input Validation Requirements

- runId must exist

Return Format Structure (JSON-like)

- results:
 - runId: UUID string
 - completionTimeTicks: integer or null (if failed/cancelled)
 - tasksPerMinute: number ≥ 0
 - avgDeliveryTimeTicks: number ≥ 0
 - energy:
 - total: number ≥ 0
 - perTask: number ≥ 0
 - congestion:
 - collisions: integer ≥ 0
 - nearMisses: integer ≥ 0
 - deadlocks: integer ≥ 0
 - batteryDeaths: integer ≥ 0
 - fairnessGini: number typically 0..1

Example Response

- results:
 - runId: 2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc
 - completionTimeTicks: 12450
 - tasksPerMinute: 18.4
 - avgDeliveryTimeTicks: 310.2
 - energy:
 - total: 9821.5
 - perTask: 49.1
 - congestion:
 - collisions: 2
 - nearMisses: 17
 - deadlocks: 1
 - batteryDeaths: 0
 - fairnessGini: 0.12

Better formatting would be to have an API for each page.

5.2.4.3 GET /results/robots/{runId}

Description and Usage Rules

- Returns per-robot aggregated stats for a run.
- Used by fairness + utilization dashboard and debugging.

Input Validation Requirements

- runId must exist

Return Format Structure (JSON-like)

- listing: array of robot stat objects
 - robotId: integer
 - navAlgorithm: one of the defined algorithms
 - tasksCompleted: integer ≥ 0
 - distanceTraveled: number ≥ 0
 - energyUsed: number ≥ 0
 - idleTicks: integer ≥ 0
 - waitTicks: integer ≥ 0
 - collisions: integer ≥ 0
 - nearMisses: integer ≥ 0
 - deadlocks: integer ≥ 0

Example Response

- listing:
 - item 1:
 - robotId: 0
 - navAlgorithm: GREEDY
 - tasksCompleted: 102
 - distanceTraveled: 812.5
 - energyUsed: 5201.0
 - idleTicks: 900
 - waitTicks: 700
 - collisions: 1
 - nearMisses: 4
 - deadlocks: 0

5.2.4.4 GET /results/compare?mapId={uuid}&policy={...}

Description and Usage Rules

- Returns aggregated results grouped by robotCount for comparison curves.
- Used to answer: "How many robots do we need?" under a given map and policy.

Input Validation Requirements

- mapId must be valid UUID and exist
- policy must be one of the defined policies

Return Format Structure (JSON-like)

- comparison:
 - mapId: UUID string
 - coordinationPolicy: string
 - rows: array of objects (grouped by robotCount)
 - robotCount: integer
 - avgCompletionTicks: number
 - avgEnergy: number

- avgCollisions: number
- avgDeadlocks: number

Example Response

- comparison:
 - mapId: b7c7f0c8-6e25-4c3c-9d08-9f2a2acb6f6e
 - coordinationPolicy: TRAFFIC_RULES
 - rows:
 - item 1: robotCount=4, avgCompletionTicks=22000, avgEnergy=7000, avgCollisions=0.2, avgDeadlocks=0.0
 - item 2: robotCount=8, avgCompletionTicks=12450, avgEnergy=9821.5, avgCollisions=2.0, avgDeadlocks=1.0
 - item 3: robotCount=12, avgCompletionTicks=12000, avgEnergy=14000, avgCollisions=6.0, avgDeadlocks=4.0

5.2.5 Logs API

5.2.5.1 API Call Signatures

- GET /logs/run/{runId}
- GET /logs/search?runId={uuid}&eventType={list}&robotId={int}&fromTick={int}&toTick={int}

5.2.5.2 GET /logs/run/{runId}

Description and Usage Rules

- Returns log events for a run (paged).
- Used by UI timeline, debugging, and replay tooling.

Input Validation Requirements

- runId must exist
- limit integer 1..200 (optional)
- offset integer ≥ 0 (optional)

Return Format Structure (JSON-like)

- listing: array of log event objects
 - tick: integer ≥ 0
 - robotId: integer or null (null means global event)
 - eventType: string from allowed set
 - pos (optional):
 - x: integer
 - y: integer
 - details: key-value dictionary
- limit: integer
- offset: integer
- total: integer

Example Response

- listing:
 - item 1:
 - tick: 1330
 - robotId: 4
 - eventType: COLLISION
 - pos: x=12, y=9
 - details: withRobot=7, policy=TRAFFIC_RULES
 - item 2:
 - tick: 1400
 - robotId: 4
 - eventType: DEADLOCK_DETECTED
 - pos: x=12, y=9
 - details: stuckTicks=120, rule=no_progress_threshold
- limit: 200
- offset: 0
- total: 2

5.2.5.3 GET

/logs/search?runId={uuid}&eventType={list}&robotId={int}&fromTick={int}&toTick={int}

Description and Usage Rules

- Filters logs for analysis:
 - by event types
 - by robotId
 - by tick range
- Used for collision analysis, deadlock audit, heatmap generation.

Input Validation Requirements

- runId required and must exist
- eventType optional; if provided must be a comma-separated list of valid event types
- robotId optional; if provided must be integer ≥ 0
- fromTick/toTick optional; if provided must be integers ≥ 0 and fromTick $\leq$ toTick

Return Format Structure (JSON-like)

- listing: array of log event objects (same as /logs/run/{runId})

Example Response

- listing:
 - item 1:
 - tick: 1330
 - robotId: 4
 - eventType: COLLISION
 - pos: x=12, y=9

- details: withRobot=7

5.2.6 Exports API (CSV)

5.2.6.1 API Call Signatures

- GET /export/results/{runId}.csv
- GET /export/robots/{runId}.csv
- GET /export/heatmap/{runId}.csv

5.2.6.2 GET /export/results/{runId}.csv

Description and Usage Rules

- Downloads run-level summary metrics as CSV.
- Used to import results into spreadsheets or plotting tools.

Input Validation Requirements

- runId must exist

Return Format Structure (CSV)

- Content-Type: text/csv
- First line is the header row; second line onward are data rows.
- Header matches results_runs.csv in the file structures section.

Example Response (first two lines)

```
run_id,map_name,robot_count,coordination_policy,workload_seed,status,completion_time_ticks
,tasks_per_minute,avg_delivery_time_ticks,total_energy,energy_per_task,collisions,near_misse
s,deadlock_count,battery_deaths,fairness_gini
2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc,narrow_aisles,8,TRAFFIC_RULES,42,COMPLETED,
12450,18.4,310.2,9821.5,49.1,2,17,1,0,0.12
```

5.2.6.3 GET /export/robots/{runId}.csv

Description and Usage Rules

- Downloads per-robot summary stats as CSV.
- Used for fairness and utilization analysis.

Input Validation Requirements

- runId must exist

Return Format Structure (CSV)

- Content-Type: text/csv
- Header matches results_robots.csv

Example Response (first two lines)

run_id,robot_id,nav_algorithm,tasks_completed,distance_traveled,energy_used,idle_ticks,wait_ticks,collisions,near_misses,deadlocks
2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc,0,GREEDY,102,812.5,5201.0,900,700,1,4,0

5.2.6.4 GET /export/heatmap/{runId}.csv

Description and Usage Rules

- Downloads tile usage counts to build heatmaps.
- Used by UI heatmap overlay and report visuals.

Input Validation Requirements

- runId must exist

Return Format Structure (CSV)

- Content-Type: text/csv
- Header columns:
 - run_id,x,y,pass_count,wait_count,congestion_score

Example Response (first two lines)

run_id,x,y,pass_count,wait_count,congestion_score
2d0fdb7c-7b5a-45e3-98fd-84c2c5dcbcdc,12,9,340,58,398

6. Critical Sections

6.1 Critical sections table of contents

Good

Title	Module	Location	Comments
CS1 Tick Order (Per-Tick Simulation Update Pipeline)	Simulation Engine (tick loop, interacts with dispatcher/robots/sensors /logging)	Main tick loop, executed once per tick (the overall sense -> decide -> coordinate -> commit -> log pipeline)	Defines the fixed order of operations each tick to ensure fairness and reproducibility. Impacts all metrics (throughput, congestion, energy, fairness).
CS2 Move Intentions & Collision/Conflict Resolution	Simulation Engine (conflict detector/resolver) + dispatcher policy enforcement	After robots generate intended moves, before movement commit in each tick	Guarantees no two robots end a tick in the same tile and prevents unsafe interactions (e.g., same-target conflicts, swap attempts).
CS3 Task Assignment (Dispatcher)	Dispatcher (task queue + assignment logic)	Start of each tick, after task spawning/queue update, before robots plan movement	Ensures tasks are assigned once, not lost/duplicated, and assigns only to eligible robots.
CS4 Traffic Rules Coordination Policy	Simulation Engine	Throughout the simulation, traffic rules are enforced for every robot moving in the simulation.	Handles traffic rules for moving through aisles, intersections, and pick-up/drop-off areas.
CS5 Tile Reservation Coordination Policy	Simulation Engine	Throughout the simulation, tile reservations are managed for every robot and are considered in navigation algorithms for every robot	Handles reservation of tiles by robots when moving throughout the warehouse map. Ensures only a single robot can reserve and enter reserved tiles.
CS6 Deadlock Detection & Recovery	Dispatcher (stuck tracking + recovery + reservations)	End of each tick, after movement is committed (so "no progress" can be measured)	Detects robots stuck for T ticks (excluding loading/unloading/idle). Recovery: log event, requeue task, clear plan/reservations, and reset robot to Idle.
CS7 Greedy Navigation Strategy	Simulation Engine	Throughout the simulation, each robot will follow this navigation strategy	Navigates towards goal by making the most greedy choice to reduce manhattan distance.

6.2 Critical Sections Breakdown

6.2.1 Critical Section 1 – Tick Order (Per-Tick Simulation Update Pipeline)

Importance & Purpose

This is a critical section because the correctness of every experiment result depends on the simulation advancing time in a consistent, repeatable order.

In particular, the tick order needs:

- Fairness: Robots should act “simultaneously” each tick, no robot should get an advantage because it is processed earlier.
- Safety correctness: Collisions and invalid moves are resolved before positions are committed.
- Data integrity: Logs and metrics must reflect the final committed state of that tick.
- Reproducibility: Given the same map + workload + robot count + algorithm configuration, results should be repeatable.

Proposed Solution

The simulation runs in discrete time steps called ticks. Each tick executes the following steps:

1. Stop checks are evaluated at the start of the tick (e.g., workload complete).
2. New pick requests may be generated for this tick and added to a task queue.
3. The dispatcher assigns tasks to eligible robots before robots decide where to move.
4. Each robot updates its knowledge from sensors, then decides its intended action for the tick (move/wait/load/unload).
5. The engine collects all robots' intended moves and applies the chosen coordination policy.
6. Remaining conflicts are resolved (collision prevention).
7. All valid movements are committed at the same time.
8. Robot state machine updates happen (loading/unloading timers, task completion).
9. Deadlock detection counters are updated, and recovery actions can be triggered.
10. Events are logged, and metrics are updated based on the committed tick outcome.
11. The tick counter increments, and the simulation proceeds to the next tick.

Flowchart of Solution

Critical Section 1 Flowchart

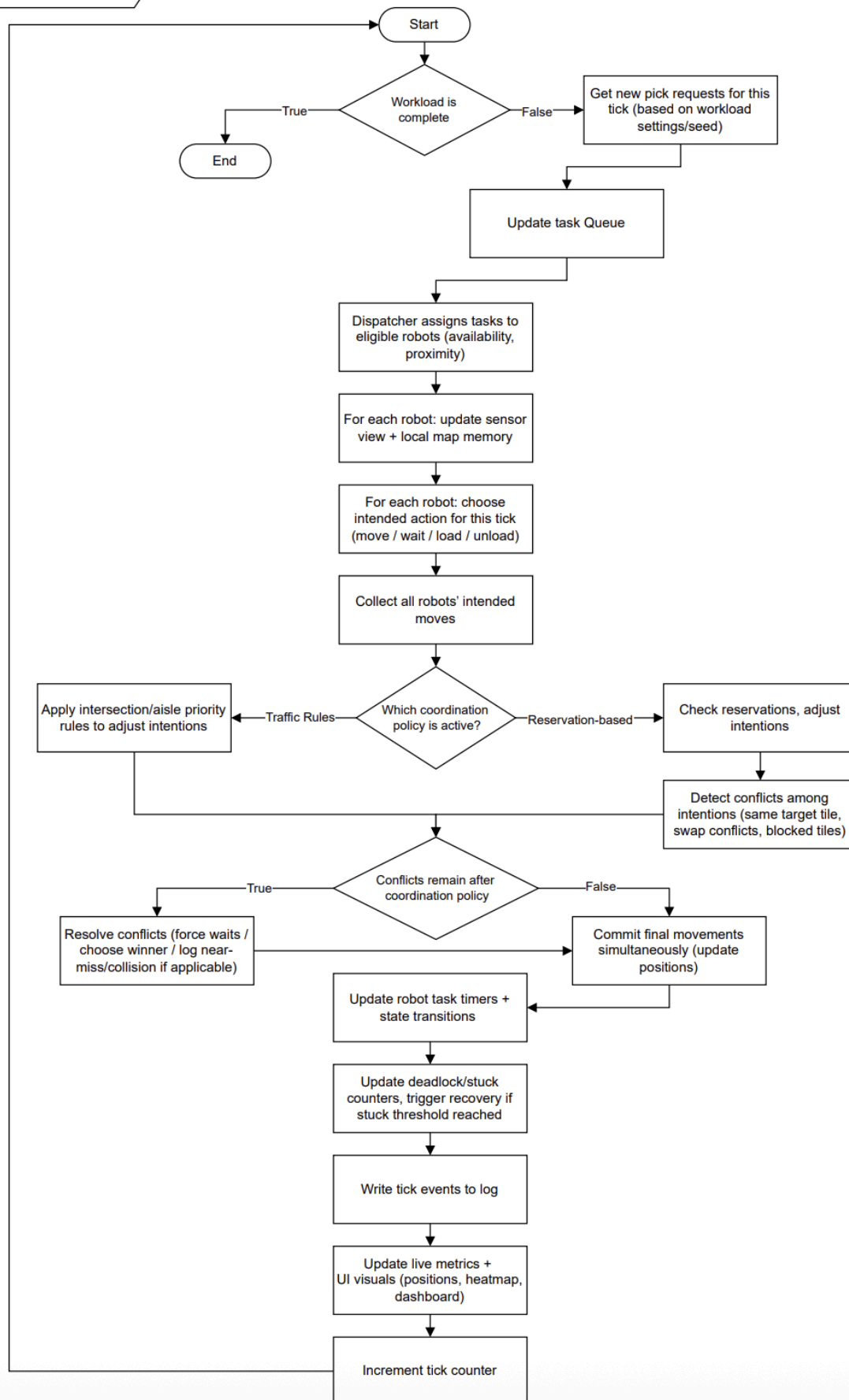


Diagram Explanation

Each robot's decision is made without any robot gaining an unfair advantage from being updated earlier in the tick. That is why robot intentions are collected first, then coordination/collision checks happen, and only then are positions committed. The dispatcher assigns tasks before robots plan movement, so movement decisions are always made with the latest task assignments. Sensor updates happen before movement planning so that strategies that depend on detected obstacles (or fog-of-war memory) are working from updated knowledge. Coordination and conflict resolution happen before movement is committed, ensuring that two robots never end up occupying the same tile at the end of a tick.

6.2.2 Critical Section 2 – Move Intentions & Collision/Conflict Resolution

Importance & Purpose

This is a critical section because the system must guarantee that two robots never end a tick in the same tile and that swapping tiles is handled safely.

Proposed Solution

Each tick, robots independently decide what they want to do next (their move intention), but the simulation engine decides what actually happens to ensure safety.

The engine uses the following approach:

1. Robots propose intentions for the tick:
 - Move to a specific adjacent walkable tile, or
 - Wait, or
 - Perform an action like loading/unloading (which implies no movement).
2. The engine performs a safety validation on each intention:
 - The intended tile must be inside the map and walkable.
 - Only one tile movement per tick.
 - Intention must follow the currently selected coordination policy (traffic rules or reservation constraints).
3. The engine detects movement conflicts:
 - Same-tile conflict: two or more robots want the same destination tile.
 - Swap conflict: robot A wants robot B's tile while robot B wants robot A's tile in the same tick.
4. The engine resolves conflicts using a fixed set of rules (so the same input produces the same output every time):
 - Choose a winner based on priority rules and a tie-breaker.
 - Robots that do not win the conflict wait this tick (no movement).

5. The engine then commits the final set of moves simultaneously, logs all collision/congestion events, and updates waiting-time metrics.

Flowchart of Solution

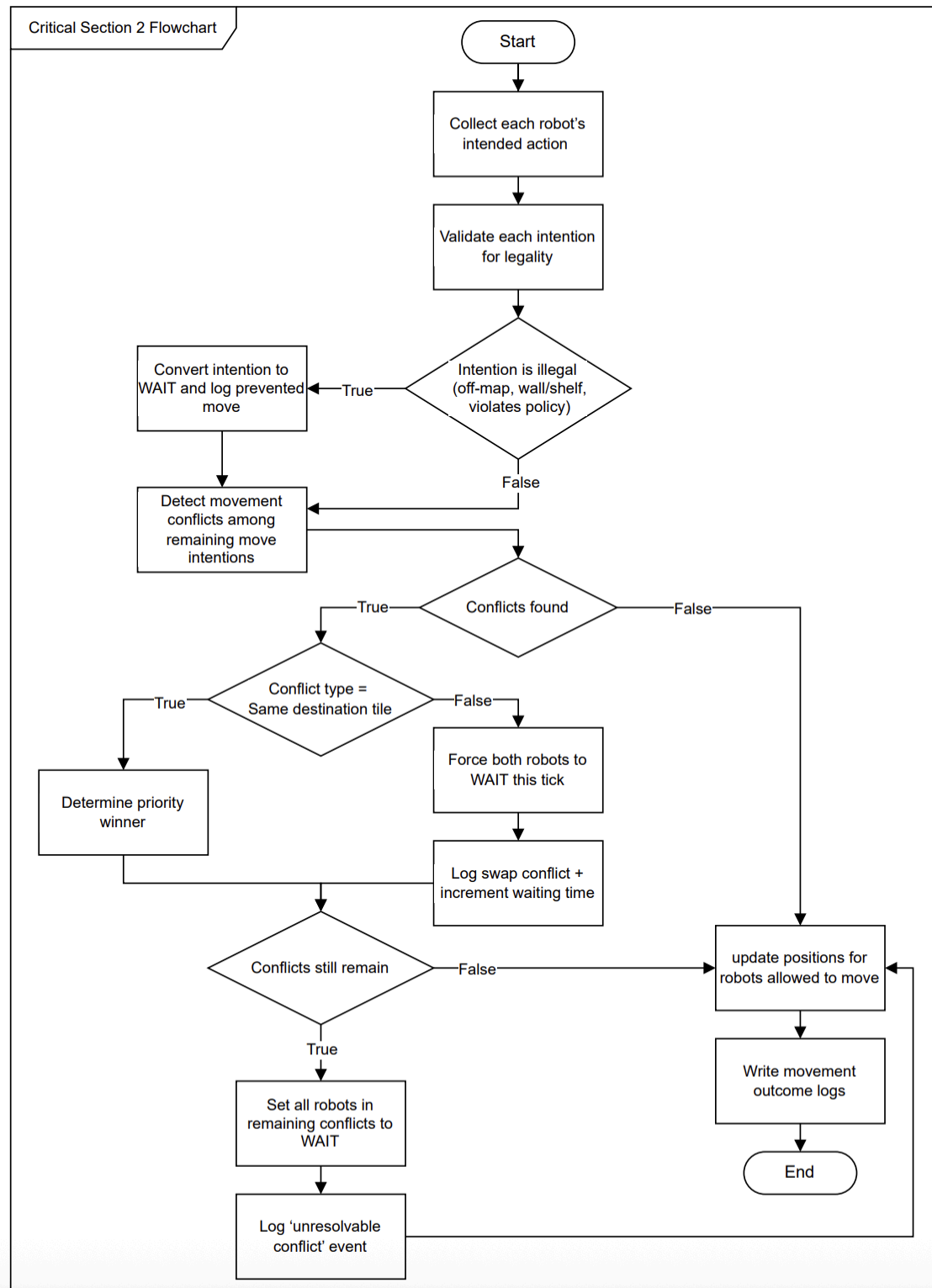


Diagram Explanation

Robots are allowed to suggest their next move, but the engine is responsible for ensuring safety and fairness. The flowchart starts by collecting all intentions first, then validating them, and only then resolving conflicts. If multiple robots want the same tile, only one robot is allowed to move, and the others wait. If two robots attempt to swap tiles in the same tick, the system forces waiting to avoid an unrealistic “pass through each other” situation.

6.2.3 Critical Section 3 – Task Assignment (Dispatcher)

Importance & Purpose

This is a critical section because if task assignment is incorrect or inconsistent, it will directly damage the reliability of our research results (throughput, congestion, fairness) and can even create failures (deadlocks, or “lost” tasks).

The dispatcher is responsible for ensuring:

- Correctness: every task is completed once, not duplicated, not lost.
- Feasibility: tasks are only assigned to robots that are available
- Fairness: work is distributed in a systematic way, so one robot isn't unfairly overloaded.
- Reproducibility: given the same config, the same assignment decisions occur.

Proposed Solution

At the start of each tick, the dispatcher runs a task-robot matching routine:

1. All new pick-and-deliver requests enter a Task Queue.
2. The dispatcher identifies which robots are currently eligible to receive a task:
 - The robot is not disabled, and is in an available state (Idle).
 - The robot does not already have an assigned task.
3. For each queued task, the dispatcher ranks eligible robots using a score based on:
 - Proximity to the pickup location,
 - Prefer robots that have done less work recently.
4. The dispatcher assigns the task to the best ranked eligible robot.
5. If no eligible robots exist for a task, it stays in the queue until later ticks.
6. When a task is assigned, the dispatcher logs the event ,and the robot changes into its “go to pickup” behavior.

Flowchart of Solution

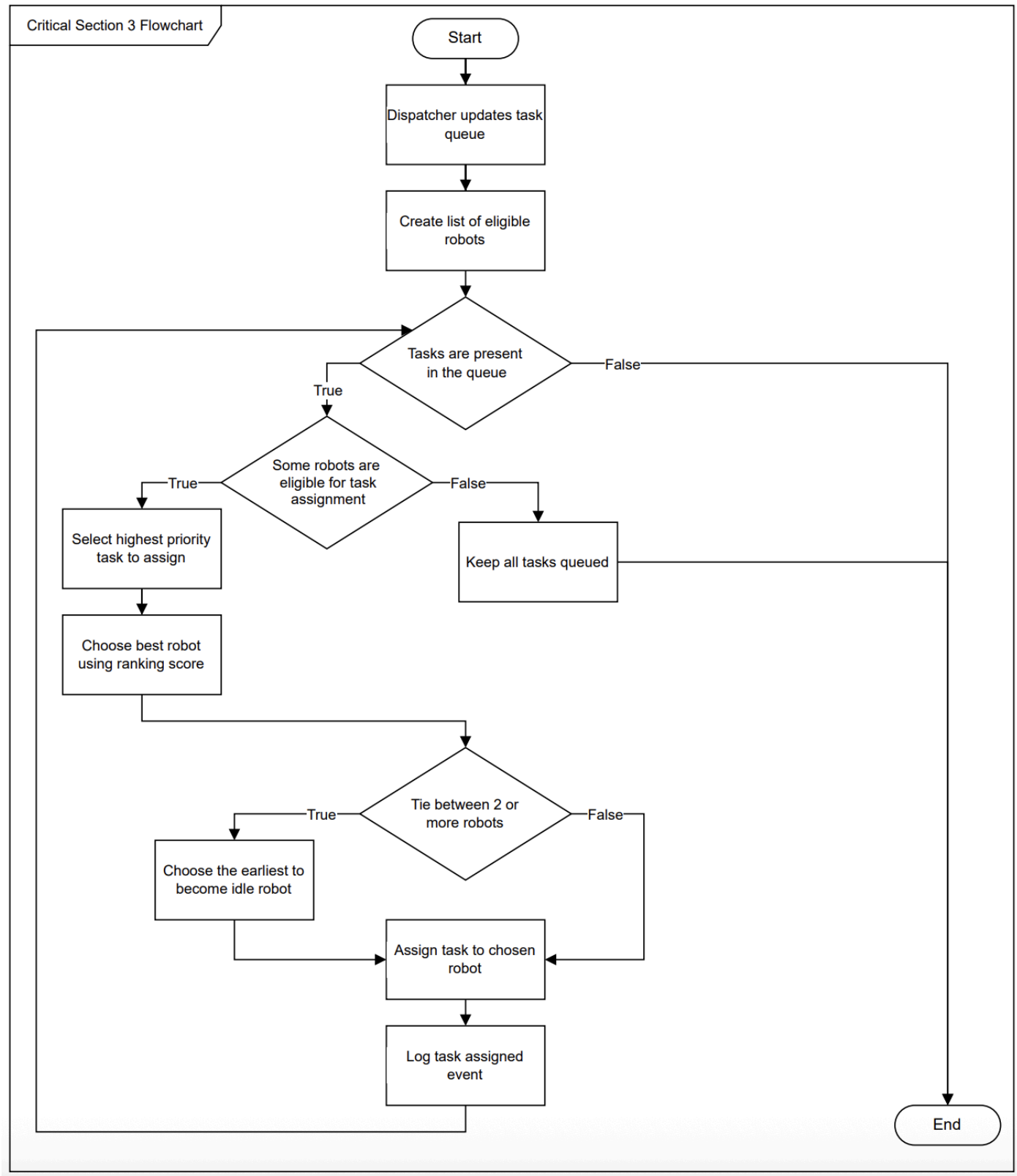


Diagram Explanation

First, tasks are collected into a queue, and the dispatcher builds an eligibility list so it never assigns a task to a robot that is already busy or otherwise unavailable. Next, tasks are considered in a consistent order, such as oldest first, which helps reproducibility and prevents starvation of older tasks. For each task, the dispatcher ranks eligible robots: closer robots reduce travel time (better throughput and less congestion). A tie-break rule is included so that if two robots appear equally good, the decision is still consistent between identical runs.

6.2.4 Critical Section 4 – Traffic Rules Coordination Policy

Importance & Purpose

The traffic rules coordination policy will be available to organize how robots coordinate movements with one another when moving around the warehouse environment. The rules in this coordination policy will help in reducing collisions and make the flow of robot traffic as smooth as possible. In particular, this policy will define the traffic rules robots will follow while driving through aisles, at intersections, and at pick-up and drop-off stations.

Proposed Solution

While driving through aisles, robots will always drive on the right side of the aisle. This will allow robot traffic to move through aisles in both directions without frequent collisions.

When entering intersections, robots will follow a first stop, first go policy where the first robot to stop proceeds through the intersection first, while all other robots wait. If multiple robots stop at an intersection at the same time, then each robot will yield to any robot to its right before moving through the intersection. However, if a robot is carrying a load, then it takes priority through the intersection first over robots not carrying a load. This will reduce task completion time and increase throughput. If multiple robots carrying a load stop at the same time, then they will yield to the robot to their right before moving through the intersection.

Finally, pick-up and drop-off stations are treated as mutually exclusive critical sections. A station may be occupied by at most one robot at a time, enforced through a centralized lock mechanism that is handled by the task dispatcher. Robots must acquire the station lock before entering the station tile and will wait if the lock is unavailable. The lock is held for the full duration of loading or unloading and is released immediately once the robot departs, ensuring safe and orderly station usage.

Flowchart of Solution

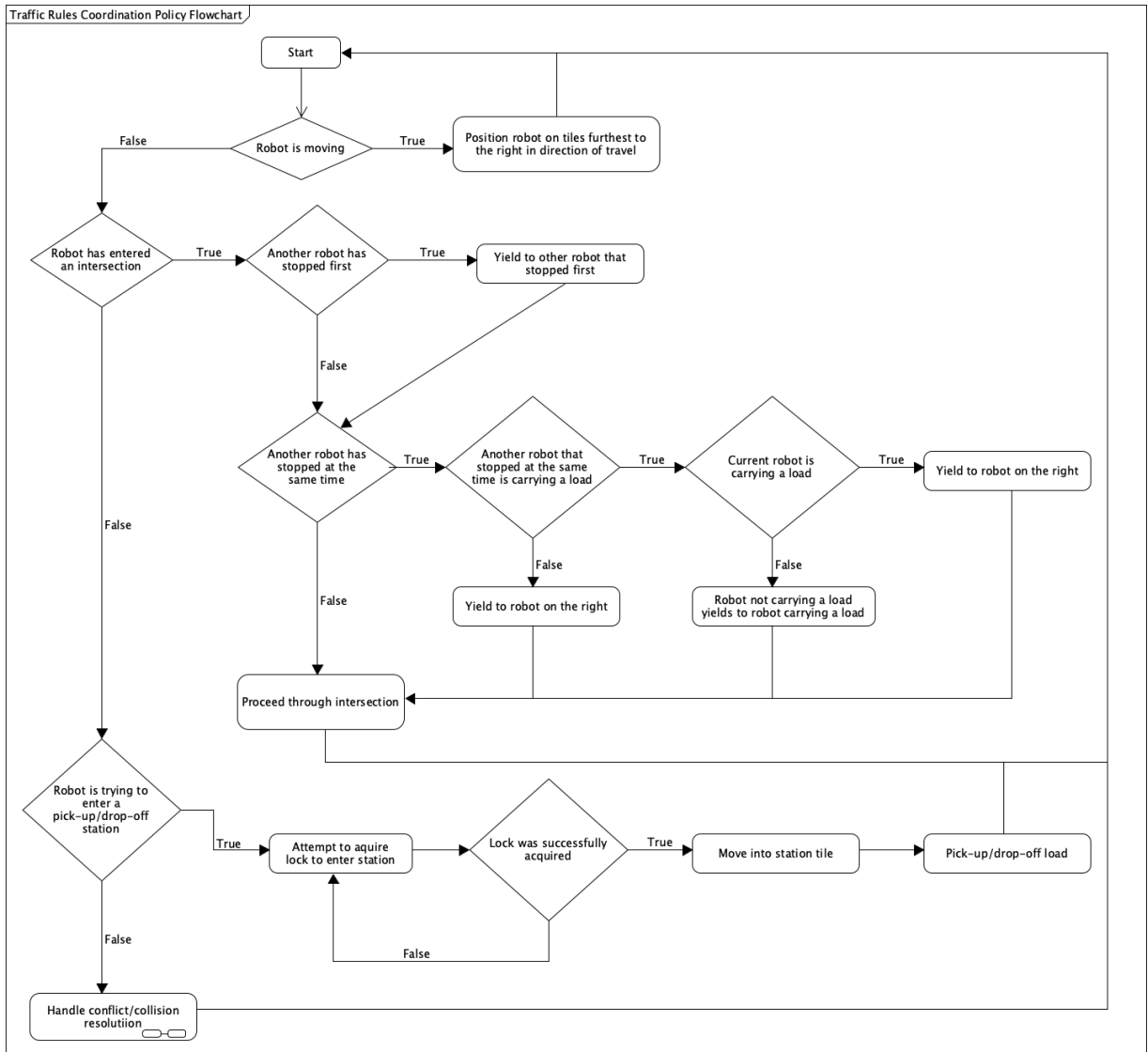


Diagram Explanation

Each robot can be moving through an aisle, trying to go through an intersection, or attempting to enter a pick-up/drop-off station. While the robot is moving, it will always position itself on the tile furthest to the right in its direction of travel. If entering an intersection, it will yield to robots that stopped first before proceeding through the intersection. If another robot stops at the same time, robots carrying loads will proceed first, followed by robots positioned to the right. When attempting to enter a pick-up or drop-off station, the robot will attempt to acquire the lock for the station before proceeding to load/unload.

6.2.5 Critical Section 5 – Tile Reservation Coordination Policy

Importance & Purpose

In high-density warehouse environments with large robot fleets, narrow aisles, or frequent intersections, congestion can lead to increased conflicts, collisions, and deadlocks. To proactively prevent these issues, the tile reservation-based coordination policy enables robots to reserve their next k movement steps in advance. By enforcing exclusive access to tiles before robots enter them, this policy reduces conflicting movement plans and improves overall predictability of robot traffic.

Proposed Solution

A tile reservation table will be centrally managed by the simulation engine for the warehouse map being simulated. Each traversable tile may be reserved by at most one robot for a given time step. After being assigned a task, a robot plans a short path segment toward its goal and attempts to reserve its next k movement steps in advance using a sliding reservation window. The value of k is specified by the user prior to running the simulation, allowing the study of trade-offs between small and large reservation horizons.

A robot may only proceed if all k requested reservations are successfully granted. If any of the requested tiles are already reserved by another robot, the robot waits in place. Reservations are released automatically as time advances or when a robot replans, ensuring that tiles are not held indefinitely. This approach proactively prevents collisions and deadlocks while maintaining adaptability in dynamic, high-density warehouse environments.

Flowchart of Solution

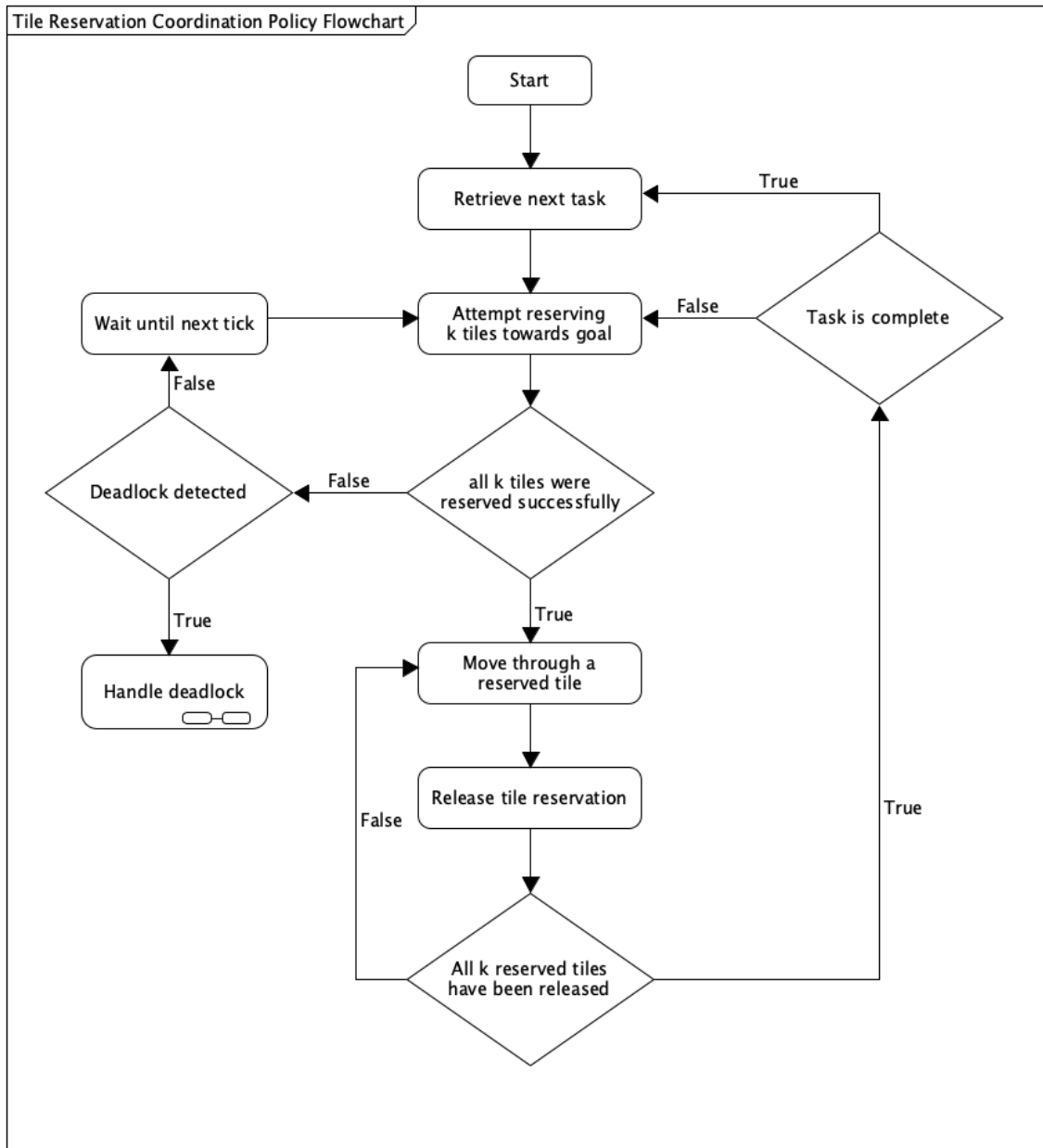


Diagram Explanation

After receiving a new task, a robot attempts to reserve its next k steps toward its goal. If the reservation fails, the robot waits until the next simulation tick and retries. If a deadlock is detected during this process, the deadlock detection algorithm resolves the situation. Once all k tiles are successfully reserved, the robot moves through them, releasing each reservation as it progresses. The robot then repeats this cycle of planning, reserving, and moving until the current task is completed, after which it is assigned a new task.

6.2.6 Critical Section 6 – Deadlock Detection & Recovery

Importance & Purpose

This is a critical section because deadlocks can cause the warehouse to stop making progress, which directly impacts our research goal: finishing the workload on time without collisions or gridlock.

If deadlocks are not handled cleanly:

- Throughput can drop to zero (simulation runs forever or hits max ticks with many tasks unfinished).
- Tasks can never get completed or duplicated (two robots think they own the same task) if recovery is inconsistent.
- Our experiment comparisons become unreliable.

Proposed Solution

The dispatcher continuously monitors each robot's ability to make progress while it is supposed to be moving toward a goal.

Deadlock definition:

A robot is considered stuck if it makes no movement progress for T consecutive ticks while it is supposed to be traveling (e.g., going to pickup, going to dropoff). To avoid false deadlocks, the stuck counter is not incremented while a robot is legitimately busy but not moving, such as loading (spending N ticks), unloading (spending M ticks), or Idle (no active task).

When a stuck state is detected (timeout reached):

1. Log a DeadlockDetected event with tick, robot id, position, and current task id.
2. If the robot has an assigned task, requeue the task so another robot can take over, preventing lost work.
3. Clear the stuck robot's movement commitments (path plan and, if applicable, any reservations it holds) so it stops blocking future decisions.
4. Reset the stuck robot to a safe state (set it to Idle)
5. Place the robot in a short recovery cooldown so it won't immediately receive a new task in the same congested area without first showing it can move again.
6. Log all recovery actions so that deadlock frequency and recovery time are measurable.

Flowchart of Solution

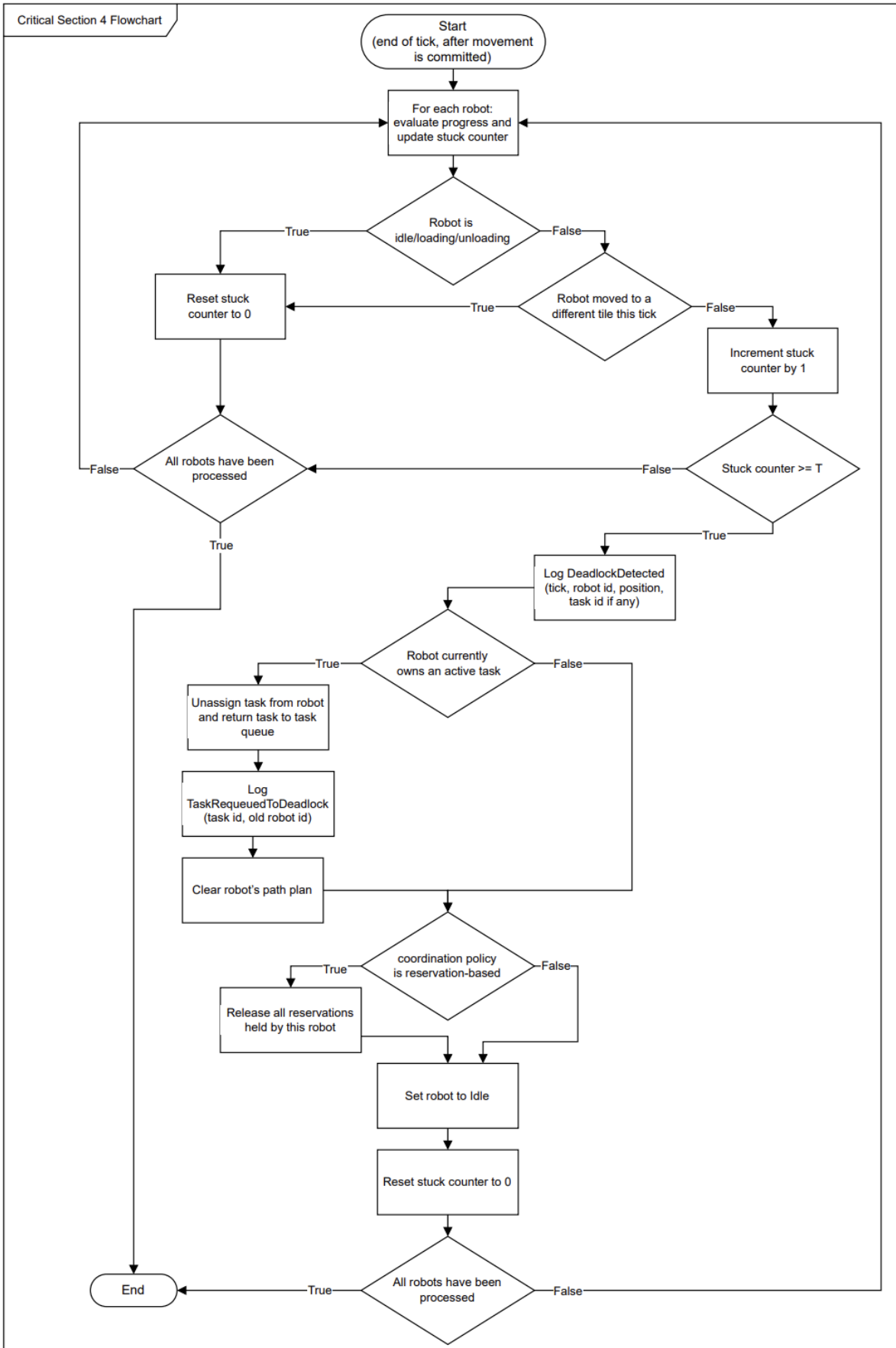


Diagram Explanation

This design treats deadlock as: no movement progress for T ticks while movement is expected. This avoids falsely flagging robot states by excluding timed actions like loading/unloading, and it also avoids flagging idle robots that are waiting for work. Deadlock recovery is intended to preserve task correctness: if a stuck robot owns a task, the task is returned to the queue so it can be completed by another robot, and the reassignment is logged. Clearing the robot's plan and releasing reservations prevents a stuck robot from continuing to hold resources that could be contributing to gridlock. Finally, resetting the robot to Idle ensures the simulation can keep progressing and prevents infinite stalls.

6.2.7 Critical Section 7 – Greedy Navigation Strategy

Importance & Purpose

All robots in the simulation must navigate the warehouse environment in order to complete assigned tasks. To do so, each robot is equipped with a navigation strategy that guides its movement toward task-specific goals, such as a shelf or packing station. One such strategy is greedy navigation, in which the robot makes purely local decisions at each simulation tick. Using sensor input, the robot evaluates its immediate neighboring tiles and selects the move that most reduces its distance to the goal while avoiding known obstacles and reserved tiles. This approach enables fast, simple navigation under partial observability, serving as an effective baseline for comparison with more advanced strategies.

Proposed Solution

At each simulation tick, the robot is aware of its current position, the position of its goal, its local sensor view, and a record of tiles it has already traversed during the current movement toward that goal. To determine its next action, the robot first identifies which adjacent tiles are traversable by filtering out tiles that contain obstacles or are otherwise blocked. It then selects its preferred movement direction by choosing the neighboring tile that most reduces its Manhattan distance to the goal. If multiple such options exist, one is selected at random to avoid deterministic bias.

If no distance-reducing moves are available, the robot considers sideways moves that maintain the same Manhattan distance. When only moves that increase the Manhattan distance are possible, the robot infers that it has entered a dead-end and initiates a backtracking behavior, returning toward a previously visited intersection to explore an alternative path. Finally, if no movement is possible due to deadlock or collision-related constraints, the robot defers to the appropriate deadlock or conflict-handling protocol.

Flowchart of Solution

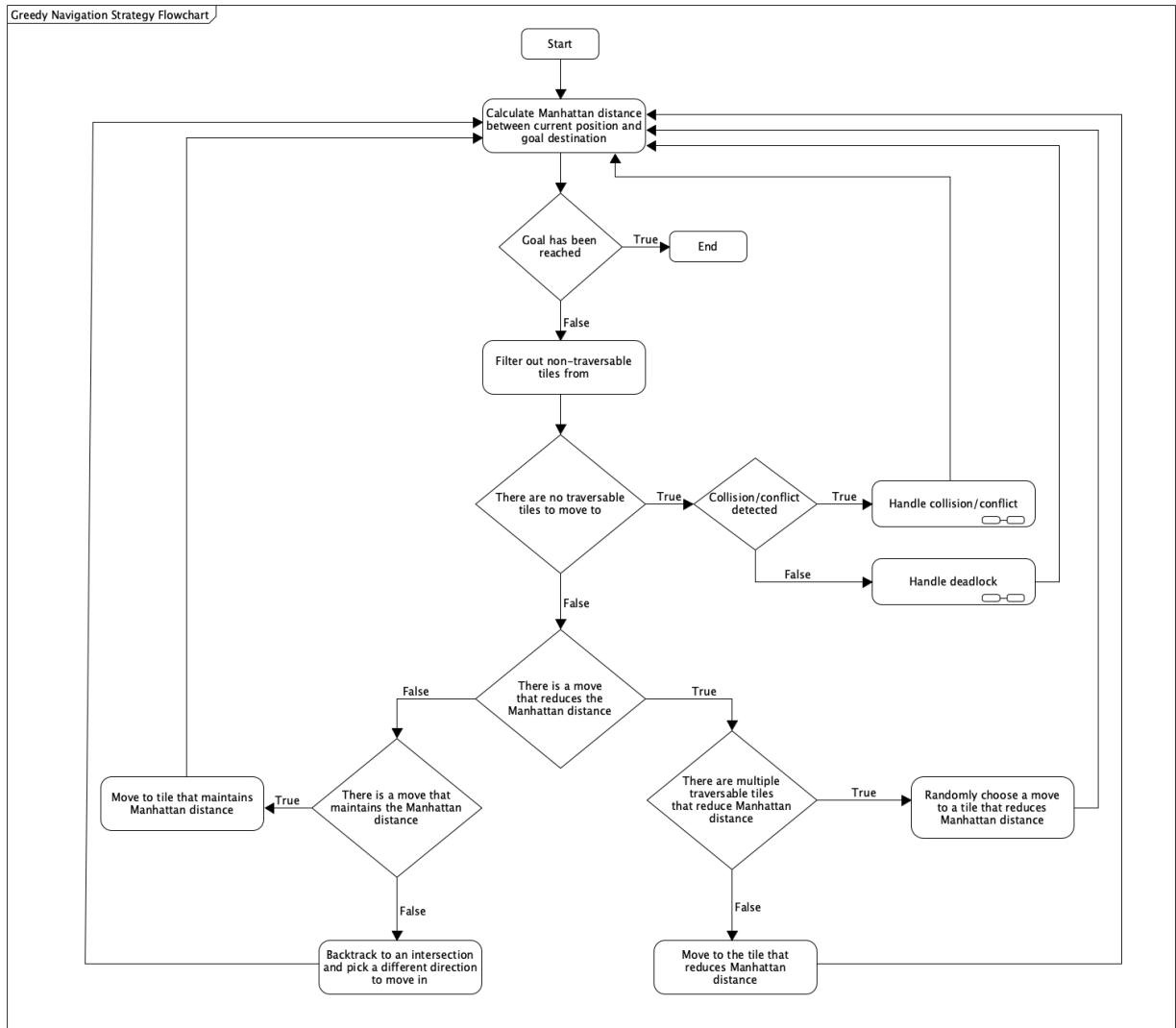


Diagram Explanation

This flowchart describes a greedy navigation strategy where the robot repeatedly evaluates its position relative to the goal using Manhattan distance. At each step, it filters out non-traversable tiles and first tries to move to a neighboring tile that reduces this distance; if multiple such moves exist, it chooses one arbitrarily. If no reducing move is available, it next considers moves that maintain the same distance, and if none exist, it backtracks to escape a dead end. The process repeats until the goal is reached or a collision or deadlock is detected and handled, ensuring the robot can recover from blocked paths rather than getting stuck.

Good

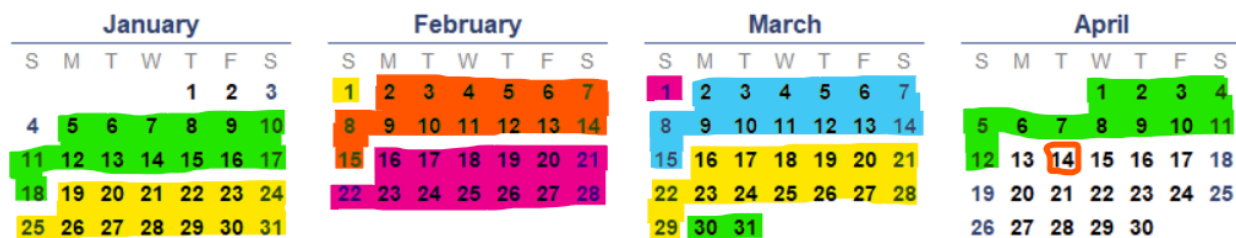
7. Effort estimation

7.1 Estimation Approach

Effort for this project will be estimated using a Prototype method, specifically the Agile workflow. We will have 2-week sprints for a total of 7 sprints, with 1-hour weekly meetings every Sunday evening. Each end of a sprint is a deliverable we want to achieve, which will be a minimal viable product that we can run, improving as we continue to develop throughout the months. We will handle our backlog with GitHub tickets (kanban board), assigning labels for each module and assigning a certain number of tickets/tasks to each team member for every sprint. Our goal is to finish the project by April 14th. The first 2 sprints will focus more on designing and documentation, and slowly start developing. Sprints 3 to 7 will be more code-intensive.

Talk about the colors.

Image 7.1.1: Sprint breakdown



Sprint 1-2 deliverables: Finalized, signed-off spec/design document, and set up dev environment in order to start working on the skeleton MVP and allow team members to learn skills/libraries they may not have.

Sprint 3 deliverable: Skeleton MVP (single robot moving randomly on screen)

Sprint 4 deliverable: Navigation MVP (robots can move using sensors)

Sprint 5 deliverable: Coordination MVP (implement job system)

Sprint 6 deliverable: Analytical MVP (metrics)

Sprint 7 deliverable: Last push MVP (optimization, clean-up, extra-features)

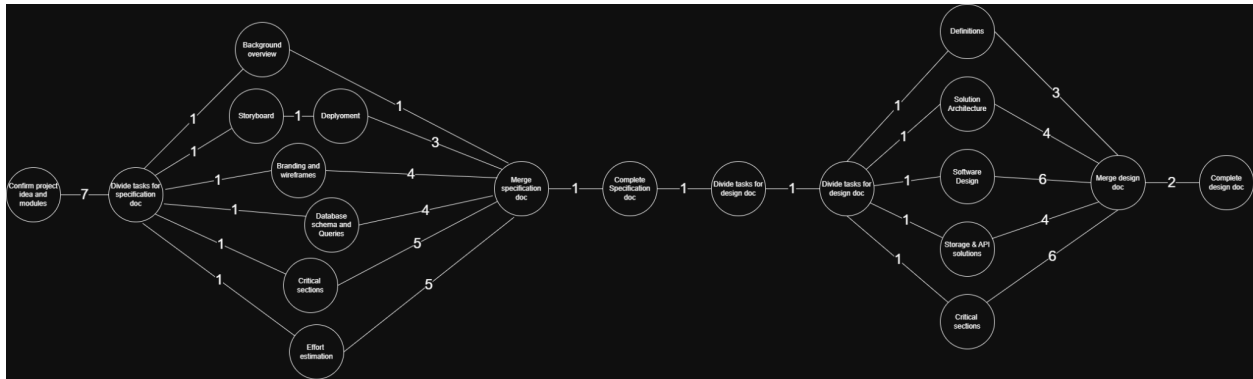
The practical shortest path would be calculated using the time given from the sprints of 14 days, so it would be around 81 days, starting after the due date of the Specification date, since no one will code before then. The optimal shortest path, calculated by the longest dependency chain from the activity graph, Graph 7.2.2, is exactly 100 days.

7.2 Effort Estimation Graph

These two graphs both start on January 5th and work in parallel.

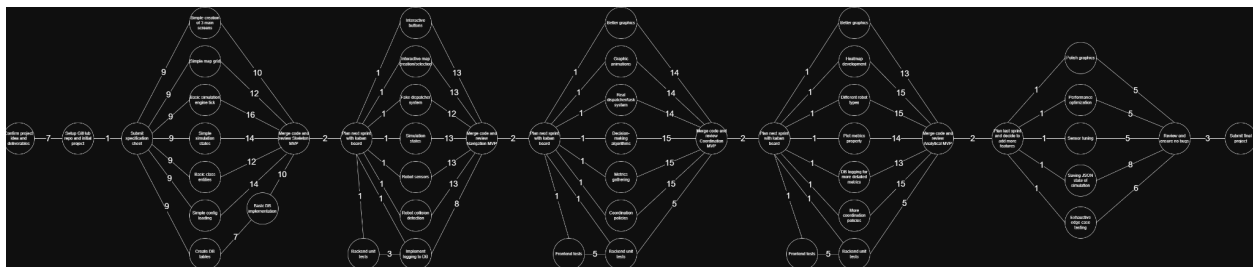
Graph 7.2.1: Activity Graph for Design and Documentation (started January 5th)

Some discussion about the shape and what is going on would be good.



[Click here to view graph in a separate window for better clarity](#)

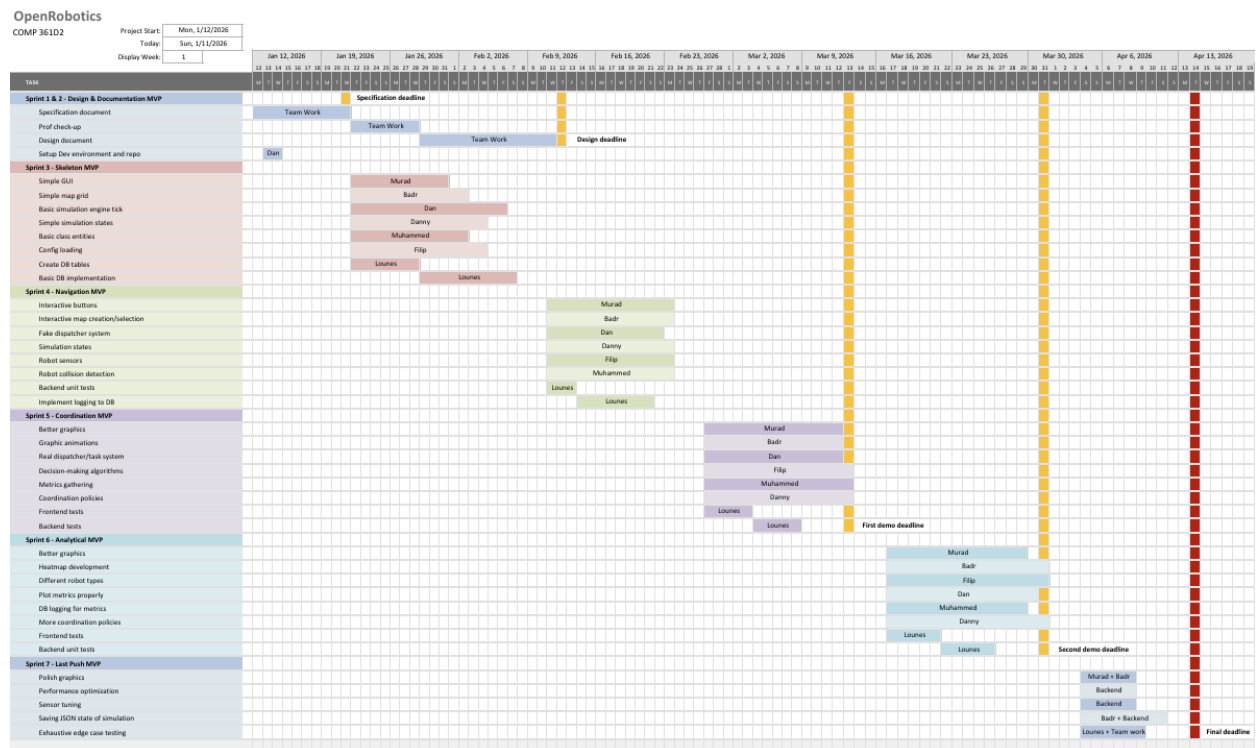
Graph 7.2.2: Activity Graph for Software Development (started January 5th)



[Click here to view graph in separate window for better clarity](#)

7.3 Gantt Chart

Same here.



[Click here to view chart for better clarity](#)

7.4 Estimated Project Cost

Since all of the software we will use is free to use, and we will package our program as a JAR file, there are no costs associated with this project. **There is a time cost and the imaginary rate per hour.**

8. Resources **On another page.**

Appendix A: Teams Contribution Table

Student Name & Student ID	Assigned Section	Contribution
Muhammad Sohail, 261142698 (Backend dev)	1 & 6	All of section 1. Critical sections 4, 5, 7, and 8.
Filip Snítíl, 261139844 (Backend dev)	2	All of section 2. Overall review.
Murad Novruzov, 261164063 (Frontend dev)	3 & 4	All of section 3. Overall Review. Formatting of doc.
Badr Zejli, 261239011 (Frontend dev)	3 & 4	All of section 4. Logo for section 3. Overall review and feedback.
Lounes Azzoun, 261181741 (QA)	5	All of section 5. Overall review.
Daniel Blackburn, 261112665 (Backend dev)	6	Section 6.1. Critical Sections 1, 2, 3, and 6. Overall review.
Dan Moraru, 261227203 (Lead/Backend dev)	7	All of section 7. Overall review. Appendix A, B.

Appendix B: Links & References

1. Gantt chart template: [Vertex42.com](https://www.vertex42.com/ExcelTemplates/simple-gantt-chart.html),
<https://www.vertex42.com/ExcelTemplates/simple-gantt-chart.html>
2. Figma draft:
<https://www.figma.com/design/GidE8ftCwCHm9NW40zcn47/App-UI?node-id=1-43>